

Relation Algebra

§15. Dynamic Programming

§15.1. Example: segmenting a list

$(T)=concat$ $T=[nil,cat]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
$(h)=list(sum)$ $h=[nil,cons(sum\times 1)]$ puts a segment's sum in front, so folding with h sums every segment
$H=partition\ list(sum)$ H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(15.1a)

x	h(x)
inl(())	[]
inr([c],[])	[c]
inr([a, b],[c])	[a + b, c]
inr([a],[b + c])	[a, b + c]

(15.1b)

$H([a, b, c],ys)\iff ys=[a + b + c]vys=[a, b + c]vys=[a + b, c]vys=[a, b, c]$ H sends $[a,b,c]$ to the sums of its four segmentations and nothing else
$T^\circ F(H)h=H$ cut once, solve the rest, put the pieces back together

(15.1c)

§15.2. Theory

$h : FB\rightarrow B$ a map, $T : FA\rightarrow A$ an F-algebra, $R : B\rightarrow B$. $H\triangleq (T)^\circ (h) : A\rightarrow B$, $M\triangleq \frac{H}{\exists} est(R)$ the problem to be solved, $(\mu X : G(X))$ as in $\triangleq \mu$.
--

(15.2a)

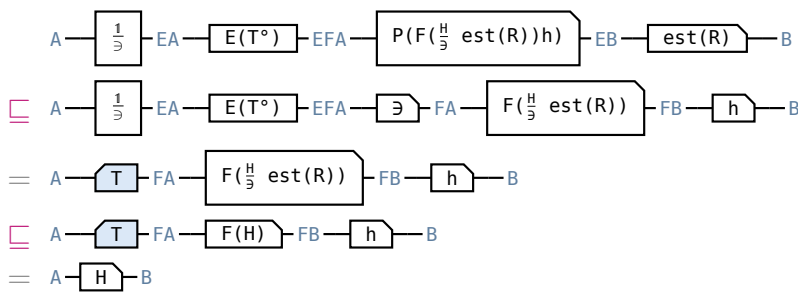
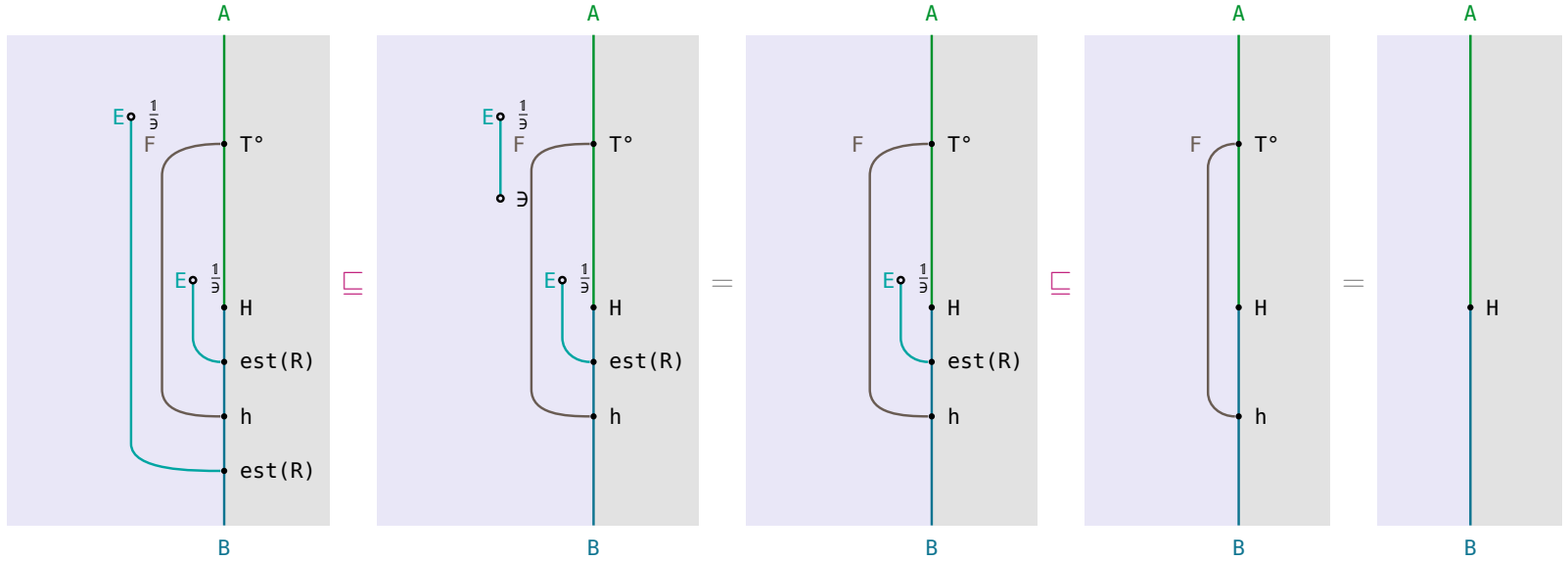
$PreservesRecip(F)\wedge Map(h)\wedge MonotonicAlg(h,R^\circ)\wedge R^\circ R^\circ \sqsubseteq R^\circ \implies (\mu X : \frac{T^\circ}{\exists} P(F(X)h)est(R))\sqsubseteq \frac{H}{\exists} est(R)$ every answer the recursion returns is an optimal one
- the body, left to right: $\frac{T^\circ}{\exists}$ takes the input apart one step every way; $F(X)$ solves each part by the recursion X; h assembles each candidate; $est(R)$ keeps a best one $\sqsubseteq M$: that never misses the optimum, though all candidates are never listed as H would h monotonic on R: a better part never makes the whole worse $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

(15.2b)

(15.2c)

$$T^\circ F(H)h=H \Rightarrow \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R)\sqsubseteq H$$

taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$) returns only what H returns



$$(9.4) P(X)\text{est}(R)\sqsubseteq \exists X \text{ at } X \triangleq F(\frac{H}{\exists} \text{est}(R))h \text{ — est-710}$$

$$\frac{T^\circ}{\exists} \exists = T^\circ \text{ — pow-laws}$$

$$M \triangleq \frac{H}{\exists} \text{est}(R) \sqsubseteq \frac{H}{\exists} \exists = H \text{ — est-up}$$

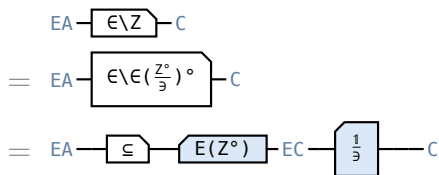
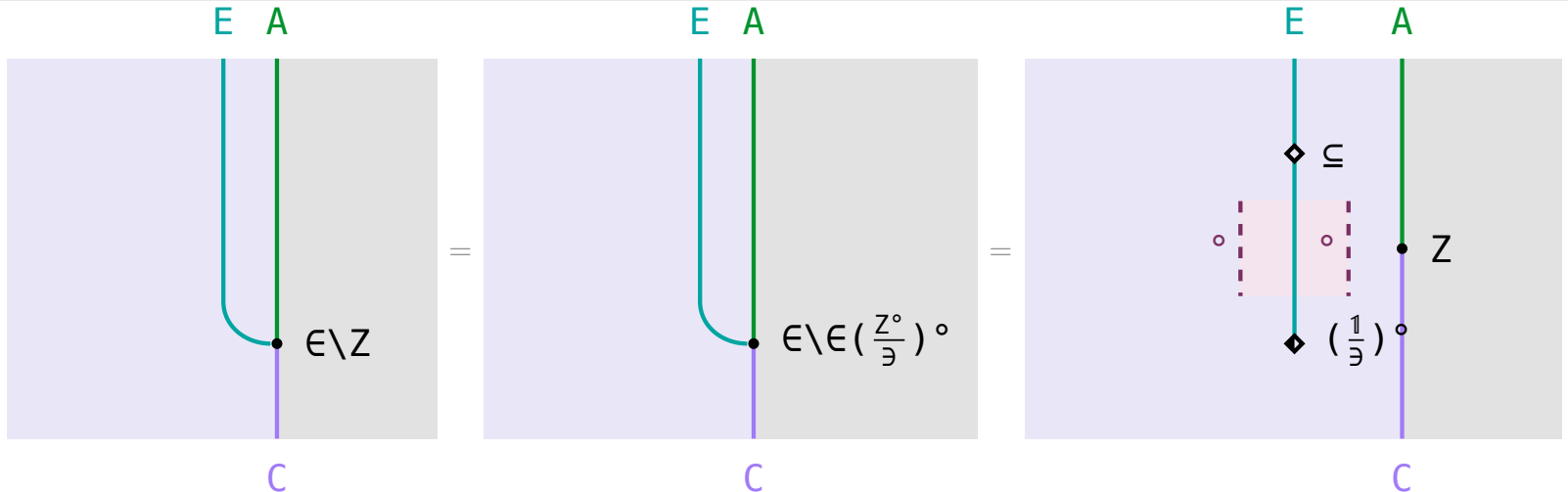
$$T^\circ F(H)h=H: H \triangleq (T)^\circ(h) \text{ and hylo-fix}$$

§15.2.1. $\epsilon \setminus$ as a composite

(15.2.1a)

$$\epsilon \setminus Z = \epsilon(\frac{Z^\circ}{\exists})^\circ$$

xs is related to c by $\epsilon \setminus Z$ exactly when $xs \sqsubseteq \Lambda(Z^\circ)(c)$, the set of every a with $a \sqsubseteq c$

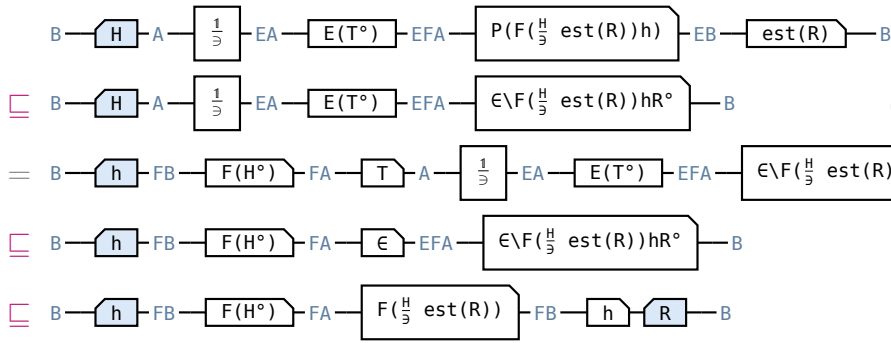
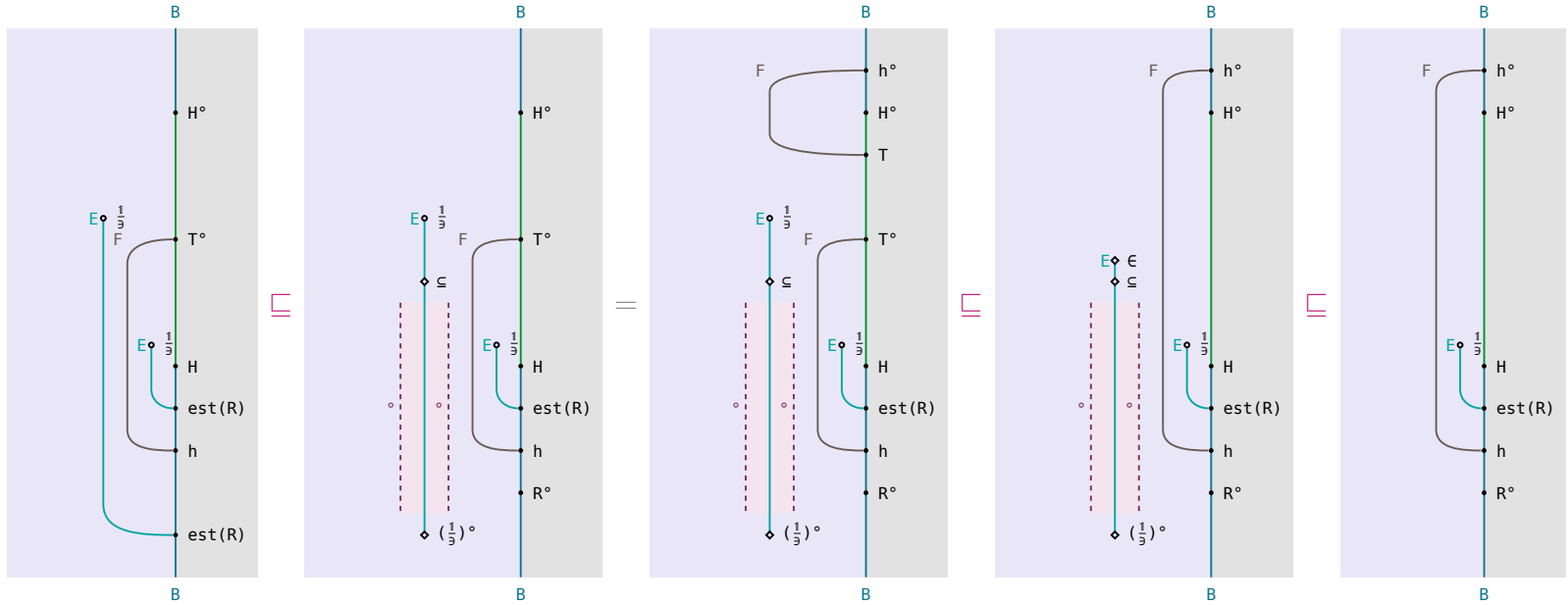


$$Z = \epsilon \Lambda(Z^\circ)^\circ: \text{the converse of } \Lambda(Z^\circ) \exists = Z^\circ \text{ — pow-laws}$$

$$X \setminus (Yf^\circ) = (X \setminus Y)f^\circ \text{ for a map } f \text{ (not a tabulated row), at the map } f \triangleq \Lambda(Z^\circ); \sqsubseteq \triangleq \epsilon \setminus \epsilon$$

PreservesRecip(F) $\wedge$ Map(h) $\wedge$ MonotonicAlg(h, R^o) $\wedge$ R^oR^o $\subseteq$ R^o $\wedge$ T^oF(H)h=H $\Rightarrow$ H^oT^o $\frac{1}{3}$ P(F($\frac{H}{3}$ est(R))h)est(R) $\subseteq$ R^o

for every **b** that **H** returns from an input, the step $\frac{1}{3}$ P(F(M)h) est(R) returns from that input only **b'** with **R** relating **b'** to **b**

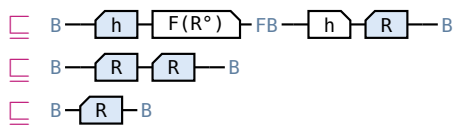
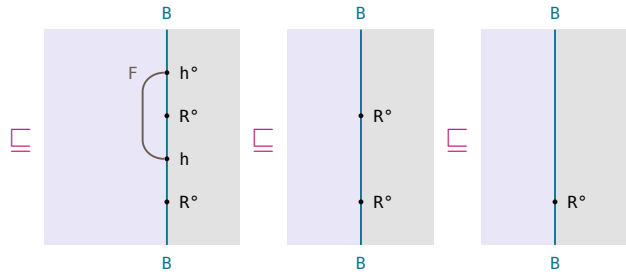


(9.4) $P(X)est(R) \subseteq E \setminus (XR^o)$ at $X \triangleq F(\frac{H}{3} est(R))h$ — est-710

$H^o = h^o F(H^o)T$: $H \triangleq (T)^o(h)$ and hylo-fix

$T \frac{1}{3} \subseteq E$: $\frac{1}{3}$ a map, $\frac{1}{3} \exists = T^o$ — pow-laws

$E(E \setminus Y) \subseteq Y$ at $Y \triangleq F(M)hR^o$ — division



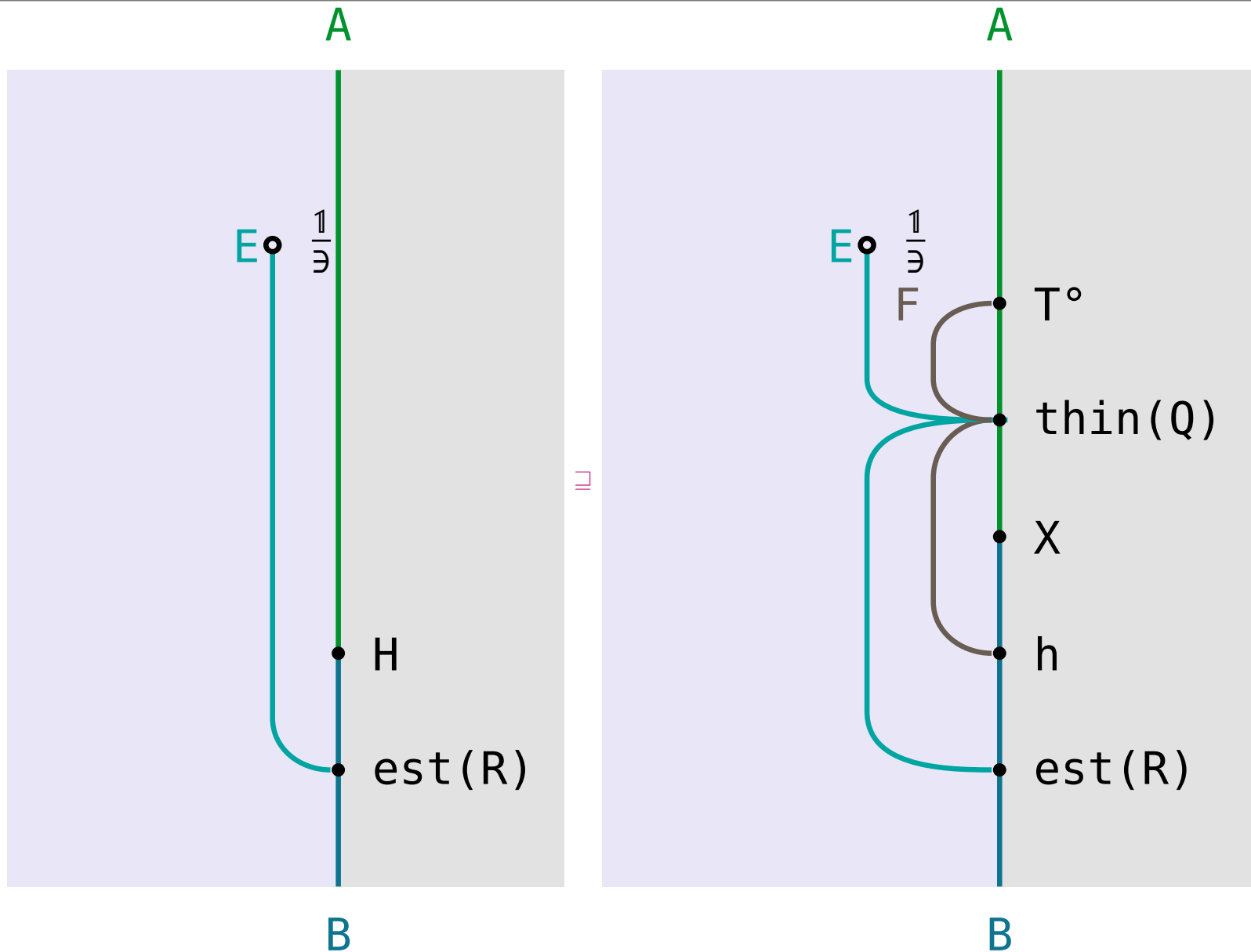
$H^o M \subseteq R^o$: $M \triangleq \frac{H}{3} est(R)$ — est-up

$h^o F(R^o)h \subseteq R^o$: h monotonic on R^o

$R^o R^o \subseteq R^o$: R transitive

$\text{PreservesRecip}(F) \wedge \text{Map}(h) \wedge \text{MonotonicAlg}(h, R^\circ) \wedge R^\circ R^\circ \subseteq R^\circ \wedge QF(H) h \subseteq F(H) h R \Rightarrow (\mu X : \frac{T^\circ}{\exists} \text{thin}(Q) P(F(X)h) \text{est}(R)) \subseteq \frac{H}{\exists} \text{est}(R)$

an optimum over everything H returns is reached by taking the input apart every way T allows, dropping the parts that can never win, solving each of the rest and keeping one optimum



$$A \xrightarrow{\frac{1}{\exists}} EA \xrightarrow{E(H)} EB \xrightarrow{\text{est}(R)} B$$

$$A \xrightarrow{\frac{1}{\exists}} EA \xrightarrow{E(T^\circ)} \text{thin}(Q) \xrightarrow{EFA} P(F(X)h) \xrightarrow{EB} \text{est}(R) \xrightarrow{} B$$

the problem to be solved, $H \triangleq (T)^\circ \circ (h) \triangleq H$

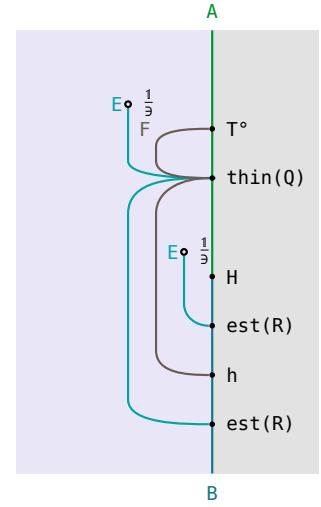
$\sqsubseteq$ h monotonic on R and Q a preorder with $QF(H) h \subseteq F(H) h R$; $\text{thin}(Q)$ as in thin-laws. Theorem 9.1 is this with the thinning step dropped — $1 \subseteq \text{thin}(Q)$. Knaster–Tarski leaves (9.1): the body at M is $\sqsubseteq M$; $M = H \cap (H^\circ \setminus R^\circ)$ splits that into (9.2) and (9.3) below — est-up. The fixed point is unique and entire when T° followed by F 's membership relation is inductive, $\frac{T^\circ}{\exists}$ finite and non-empty, R connected

circuit

Hinze–Marsden

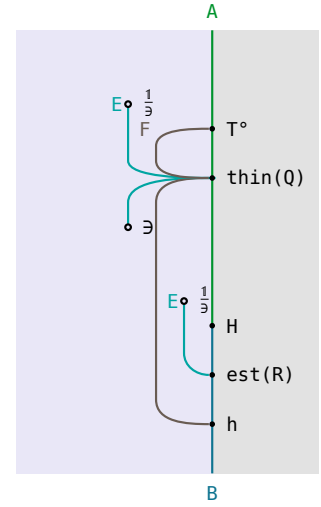
$$A \xrightarrow{\frac{1}{\exists}} EA \xrightarrow{E(T^\circ)} \text{thin}(Q) \xrightarrow{EFA} P(F(\frac{H}{\exists} \text{est}(R))h) \xrightarrow{EB} \text{est}(R) \xrightarrow{} B$$

(9.2)



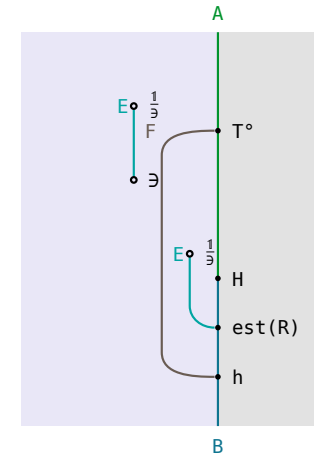
$$\sqsubseteq A \xrightarrow{\frac{1}{\exists}} EA \xrightarrow{E(T^\circ)} \text{thin}(Q) \xrightarrow{EFA} \exists \text{FA} \xrightarrow{F(\frac{H}{\exists} \text{est}(R))} FB \xrightarrow{h} B$$

(9.4) $P(X)\text{est}(R) \sqsubseteq \exists X \text{ at } X \triangleq F(\frac{H}{\exists} \text{est}(R))h \text{ — est-710}$



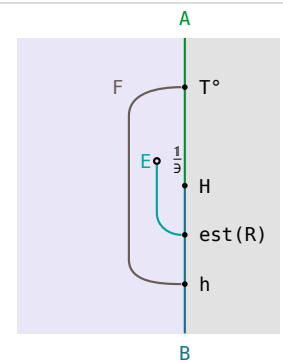
$$\sqsubseteq A \xrightarrow{\frac{1}{\exists}} EA \xrightarrow{E(T^\circ)} EFA \xrightarrow{\exists} FA \xrightarrow{F(\frac{H}{\exists} \text{est}(R))} FB \xrightarrow{h} B$$

$\text{thin}(Q)\exists \sqsubseteq \exists \text{ — thin-laws}$



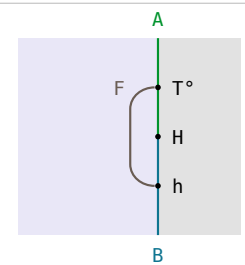
$$= A \xrightarrow{\frac{T^\circ}{\exists}} FA \xrightarrow{F(\frac{H}{\exists} \text{est}(R))} FB \xrightarrow{h} B$$

$\frac{T^\circ}{\exists} \exists = T^\circ \text{ — pow-laws}$



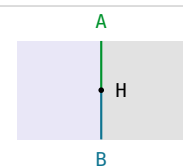
$$\sqsubseteq A \xrightarrow{\frac{T}{H}} FA \xrightarrow{F(H)} FB \xrightarrow{h} B$$

$M \sqsubseteq \frac{H}{\exists} \exists = H \text{ — est-up}$



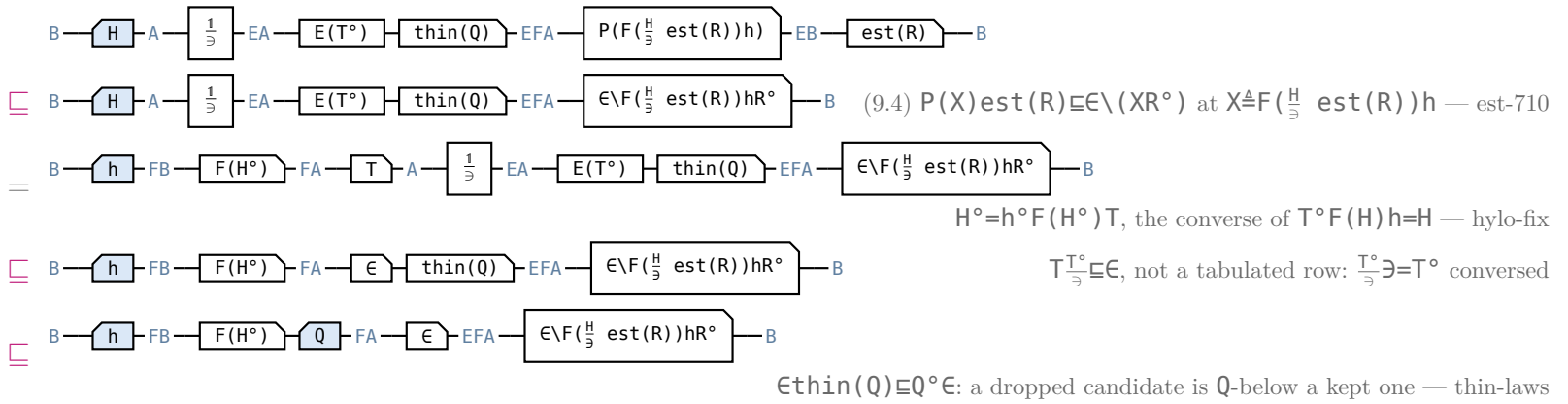
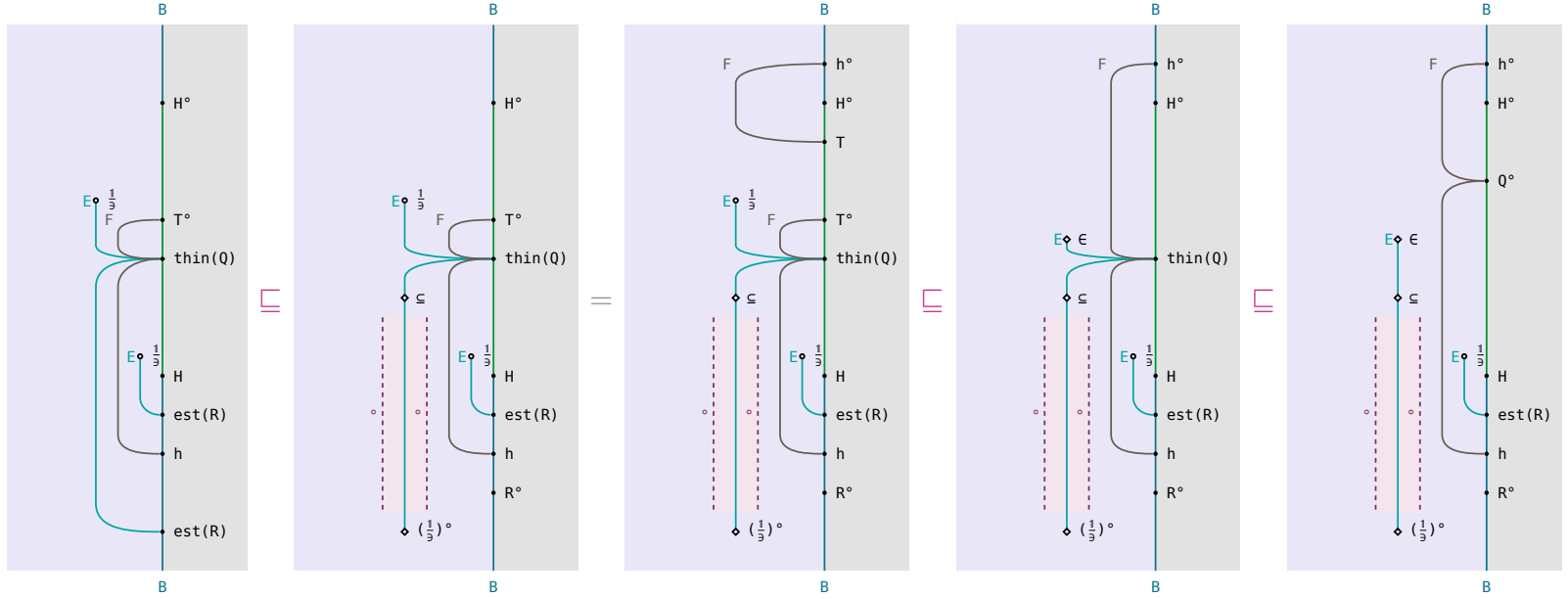
$$= A \xrightarrow{\frac{H}{T}} B$$

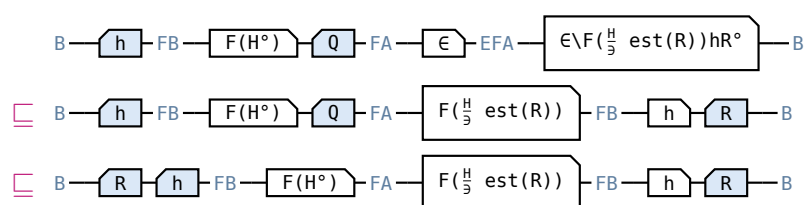
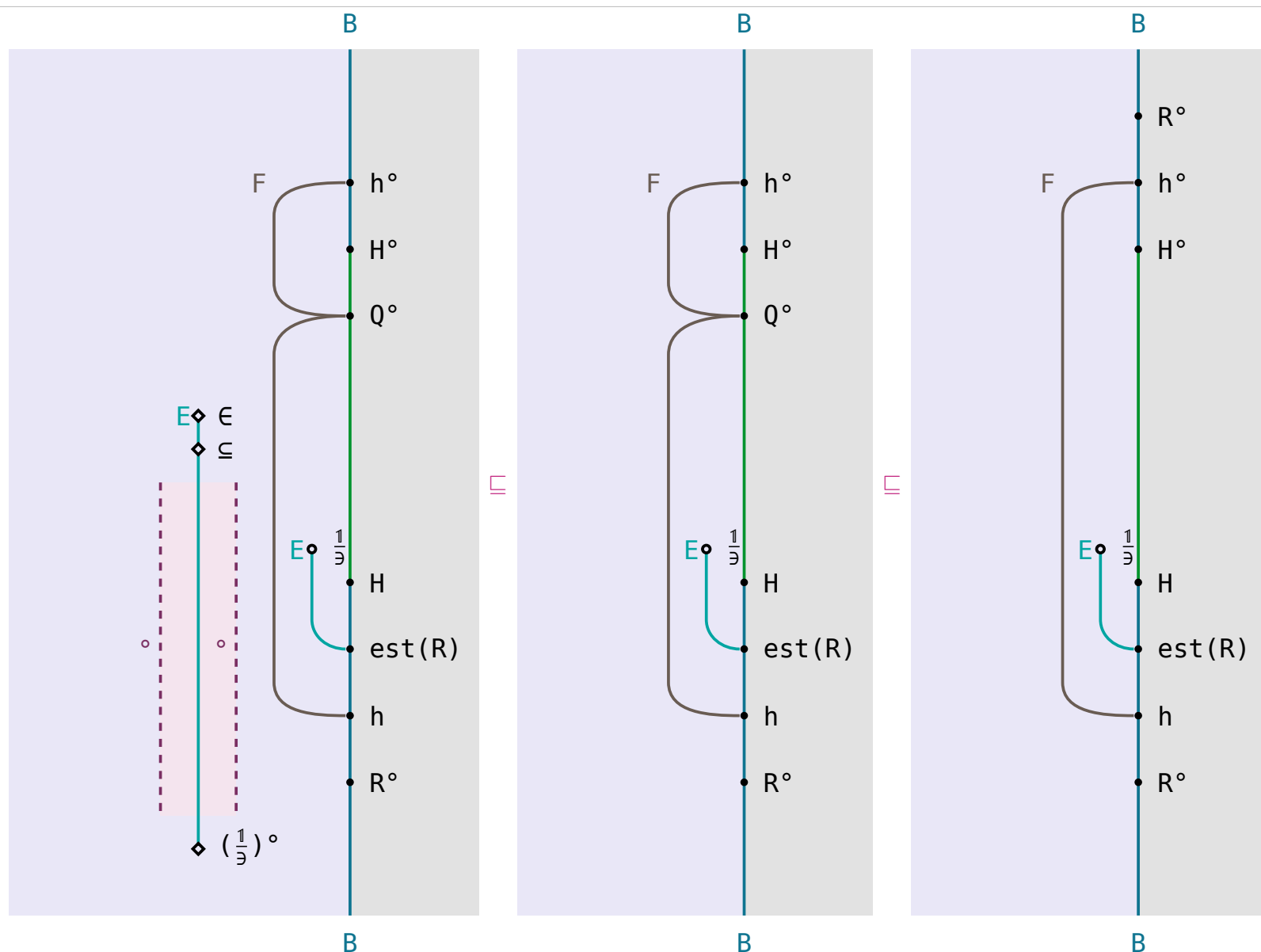
$T^\circ F(H)h = H: H \triangleq (T)^\circ(h) \text{ and hylo-fix}$



$\text{PreservesRecip}(F) \wedge \text{Map}(h) \wedge \text{MonotonicAlg}(h, R^\circ) \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge T^\circ F(H)h = H \wedge QF(H)h \sqsubseteq F(H)hR \implies H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) P(F(\frac{H}{\exists} \text{est}(R))h) \text{est}(R) \sqsubseteq R^\circ$

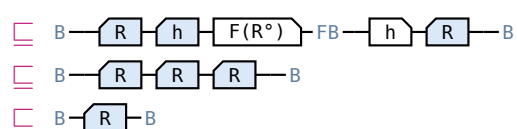
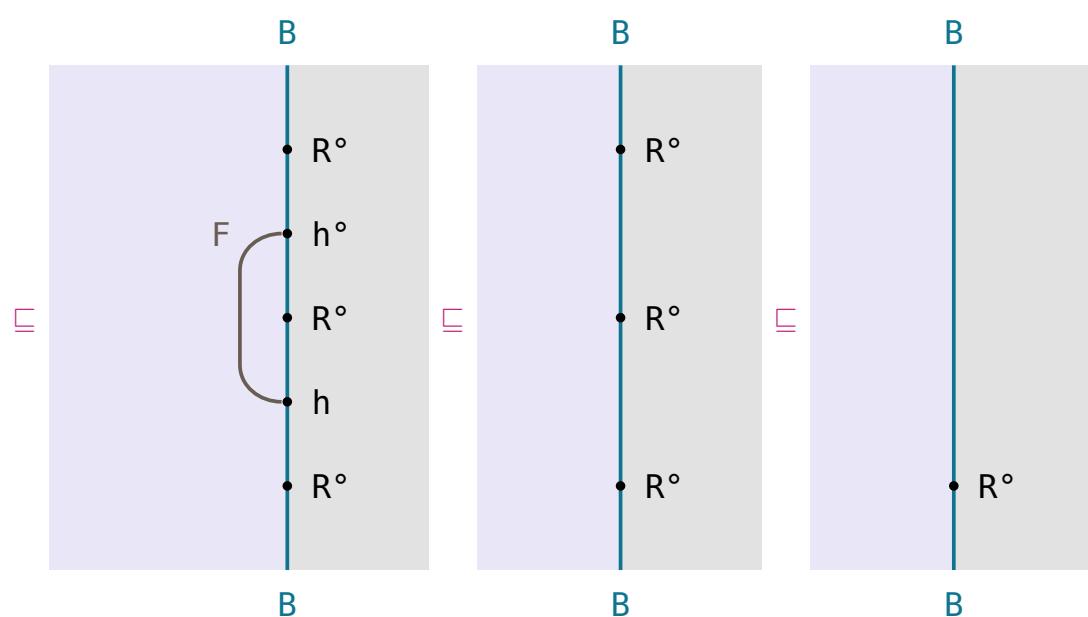
H° followed by the body at M is $\sqsubseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input





$E(E \setminus Y) \subseteq Y$, not a tabulated row: division cancels

$QF(H)h \subseteq F(H)hR$ conversed — the hypothesis on Q



$H^\circ M \subseteq R^\circ$ under F — est-up

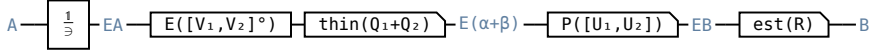
$h^\circ F(R^\circ)h \subseteq R^\circ$: h monotonic on R , shunted — the hypothesis on h

R transitive, twice

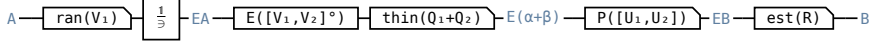
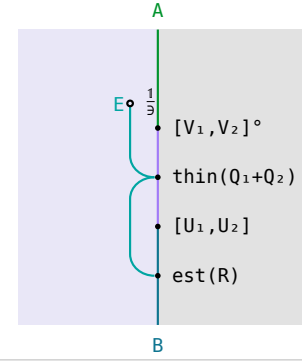
$$V_2 \quad V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{\exists} \text{thin}(Q_1 + Q_2) P([U_1, U_2]) \text{est}(R) = \text{ran}(V_1) \frac{V_1^\circ}{\exists} \text{thin}(Q_1) P(U_1) \text{est}(R) \cup \text{ran}(V_2) \frac{V_2^\circ}{\exists} \text{thin}(Q_2) P(U_2) \text{est}(R)$$

when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches

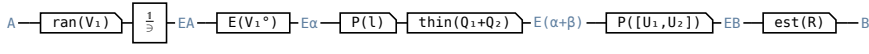
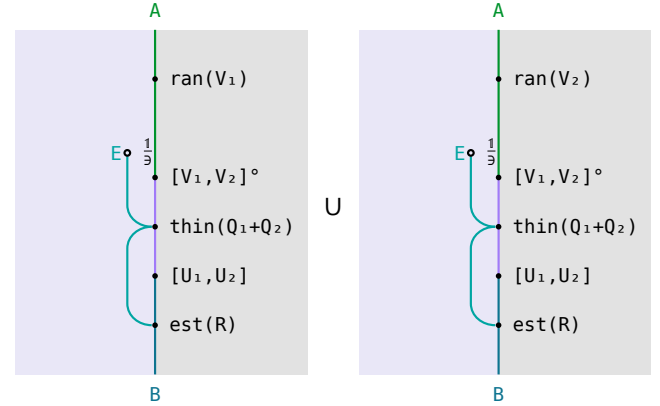
circuit



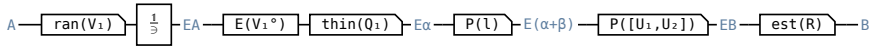
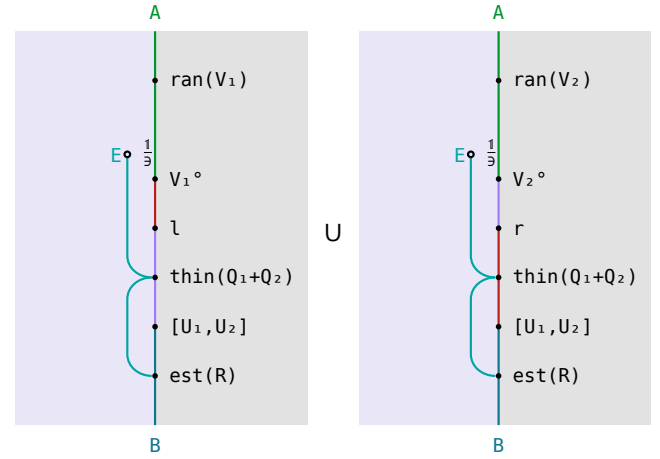
Hinze–Marsden



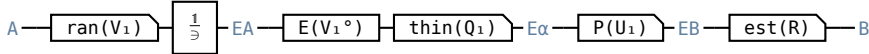
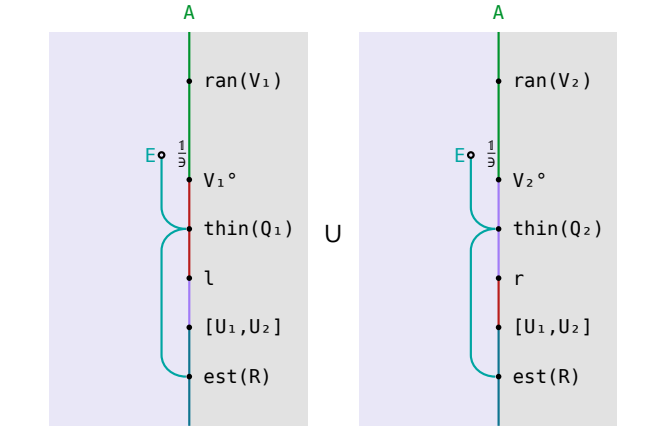
off $\text{ran } V_1 \cup \text{ran } V_2$ the set is empty and $\text{est}(R)$ of it is nothing; not a tabulated row



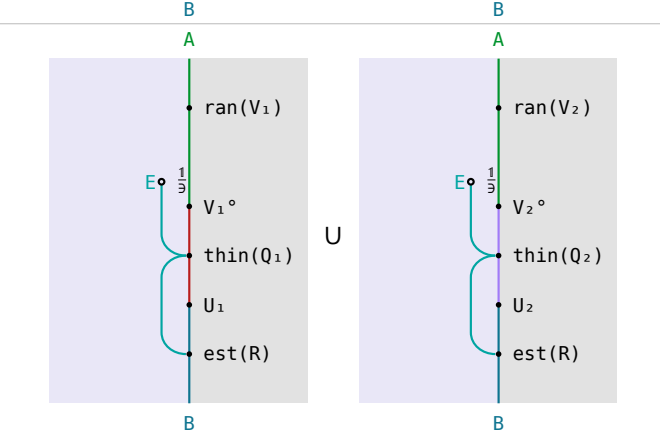
Exercise 9.5: on $\text{ran } V_1$, $\frac{[V_1, V_2]^\circ}{\exists} = \frac{V_1^\circ}{\exists} P(\text{inl})$, as V_1, V_2 have disjoint ranges



Exercise 9.5: $P(\text{inl}) \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) P(\text{inl})$



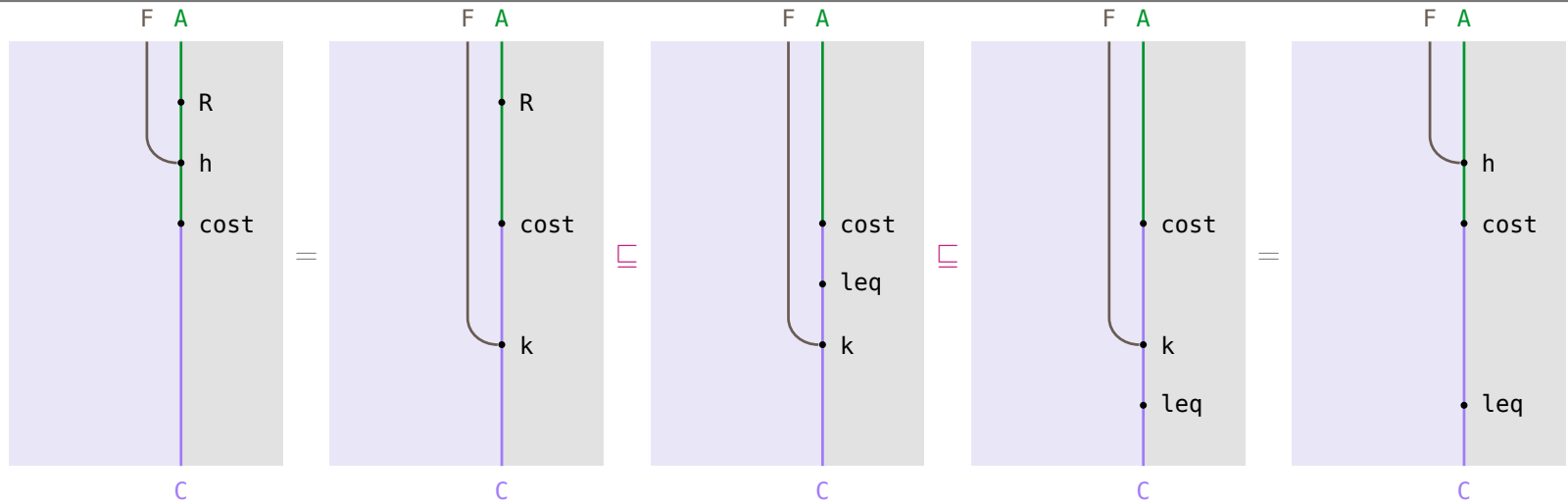
P a relator and $\text{inl}[U_1, U_2] = U_1$



(15.2.1h)

 $\text{Map}(\text{cost}) \wedge R = \text{cost} \text{ leq } \text{cost}^\circ \wedge h \text{ cost} = F(\text{cost})k \wedge F(\text{leq})k \sqsubseteq k \text{ leq} \implies \text{MonotonicAlg}(h, R)$

if R compares two values by comparing their **costs** under $\leq$, and h then **cost** equals $F(\text{cost})$ then a k monotonic on $\leq$, then h is monotonic on R



$$\begin{aligned}
 & FA \text{---} \boxed{F(R)} \text{---} h \text{---} A \text{---} \boxed{\text{cost}} \text{---} C \\
 = & FA \text{---} \boxed{F(R)} \text{---} \boxed{F(\text{cost})} \text{---} FC \text{---} \boxed{k} \text{---} C \\
 \sqsubseteq & FA \text{---} \boxed{F(\text{cost})} \text{---} \boxed{F(\text{leq})} \text{---} FC \text{---} \boxed{k} \text{---} C \\
 \sqsubseteq & FA \text{---} \boxed{F(\text{cost})} \text{---} FC \text{---} \boxed{k} \text{---} \boxed{\text{leq}} \text{---} C \\
 = & FA \text{---} h \text{---} A \text{---} \boxed{\text{cost}} \text{---} \boxed{\text{leq}} \text{---} C
 \end{aligned}$$

$F(R)h \sqsubseteq hR$ iff $F(R)h \text{ cost} \sqsubseteq h \text{ cost} \leq$: definition of R and shunting
 assumption $h \text{ cost} = F(\text{cost})k$
 $R \text{ cost} \sqsubseteq \text{cost} \leq$, as cost is a map; functors
 assumption $F(\leq)k \sqsubseteq k \leq$: k monotonic on $\leq$
 assumption $h \text{ cost} = F(\text{cost})k$

the condition

$F(R)h \sqsubseteq hR$

Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{ cost} = F(\text{cost})k$, $F(\leq)k \sqsubseteq k \leq$

$F(Rn(H^\circ H))h \sqsubseteq hR$

Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{ cost} = F(\langle \text{cost}, H^\circ \rangle)k$,
 $F(\leq \times 1)k \sqsubseteq k \leq$, H° simple;

$QF(H)h \sqsubseteq F(H)hR$ at $Q \triangleq F(U, V)$

Proposition 9.4, U, V preorders, $F(U, R)h \sqsubseteq hR$, $VH \sqsubseteq HR$

how it is discharged

monotonicity when the cost is itself a fold with a step k
 monotonic on $\leq$

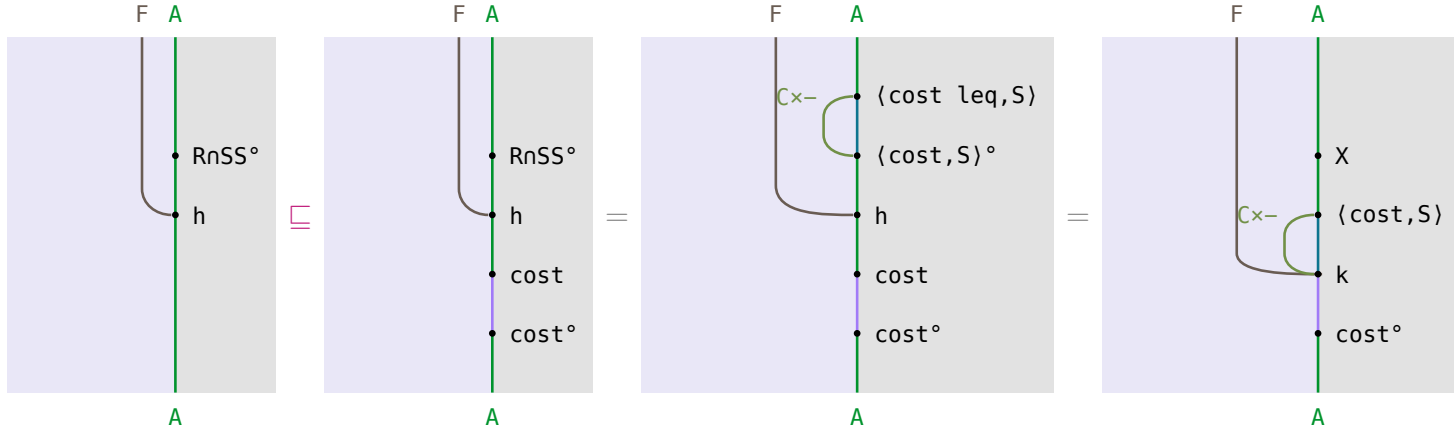
monotonicity **in context**: k may also read the input the
 part was built from

both conditions at once, split along the two arguments of a
 bifunctor

(15.2.1i)

$\text{Map}(\text{cost}) \wedge \text{Simple}(S) \wedge R = \text{cost} \text{ leq } \text{cost}^\circ \wedge h \text{cost} = F(\langle \text{cost}, S \rangle) k \wedge F(\text{leq} \times 1) k \sqsubseteq k \text{ leq} \Rightarrow F(RnSS^\circ) h \sqsubseteq h R$

improving each part by R within its S -context, then assembling by h , is below hR

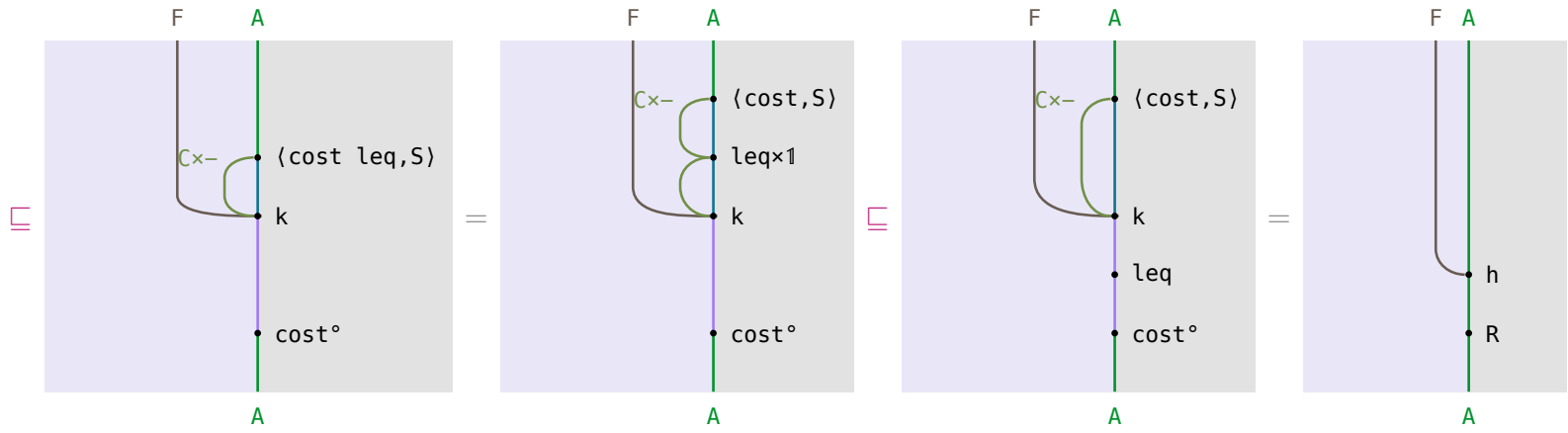


$$\begin{aligned}
 & FA \rightarrow \boxed{F(RnSS^\circ)} \rightarrow h \rightarrow A \\
 \sqsubseteq & FA \rightarrow \boxed{F(RnSS^\circ)} \rightarrow h \rightarrow A \rightarrow \boxed{\text{cost}} \rightarrow C \rightarrow \boxed{\text{cost}} \rightarrow A \\
 = & FA \rightarrow \boxed{F(\langle \text{cost leq}, S \rangle \langle \text{cost}, S \rangle^\circ)} \rightarrow h \rightarrow A \rightarrow \boxed{\text{cost}} \rightarrow C \rightarrow \boxed{\text{cost}} \rightarrow A \\
 = & FA \rightarrow \boxed{F(X)} \rightarrow \boxed{F(\langle \text{cost}, S \rangle)} \rightarrow F(C \times B) \rightarrow k \rightarrow C \rightarrow \boxed{\text{cost}} \rightarrow A
 \end{aligned}$$

shunting: cost a map, so $1 \sqsubseteq \text{cost} \text{ cost}^\circ$ — triple-chains

products: $RnSS^\circ = \langle \text{cost leq}, S \rangle \langle \text{cost}, S \rangle^\circ$ at $R = \text{cost leq cost}^\circ$ — $\triangleq \times$

assumption on cost : $h \text{ cost} = F(\langle \text{cost}, S \rangle) k$



$$\begin{aligned}
 \sqsubseteq & FA \rightarrow \boxed{F(\langle \text{cost leq}, S \rangle)} \rightarrow F(C \times B) \rightarrow k \rightarrow C \rightarrow \boxed{\text{cost}} \rightarrow A \\
 = & FA \rightarrow \boxed{F(\langle \text{cost}, S \rangle)} \rightarrow \boxed{F(\text{leq} \times 1)} \rightarrow F(C \times B) \rightarrow k \rightarrow C \rightarrow \boxed{\text{cost}} \rightarrow A \\
 \sqsubseteq & FA \rightarrow \boxed{F(\langle \text{cost}, S \rangle)} \rightarrow F(C \times B) \rightarrow k \rightarrow \boxed{\text{leq}} \rightarrow C \rightarrow \boxed{\text{cost}} \rightarrow A \\
 = & FA \rightarrow \boxed{h} \rightarrow \boxed{R} \rightarrow A
 \end{aligned}$$

S simple, so $\langle \text{cost}, S \rangle$ simple: $\langle \text{cost}, S \rangle^\circ \langle \text{cost}, S \rangle \sqsubseteq 1$

products; functors: $\langle \text{cost leq}, S \rangle = \langle \text{cost}, S \rangle (\text{leq} \times 1)$ — bdm-prod-laws, relator-laws

assumption on k : $F(\text{leq} \times 1) k \sqsubseteq k \text{ leq}$

assumption on cost read backwards, then $R = \text{cost leq cost}^\circ$

$G(U, R)h \sqsubseteq hR \wedge Vh \sqsubseteq HR \implies G(U, V)G(1, H)h \sqsubseteq G(1, H)hR$ thinning the parts by U in the first argument and by V in the second ($G(U, V)$), then solving by H and assembling by h , gives only what solving and assembling and then improving by R gives ($G(1, H)hR$)	
circuit	Hinze–Marsden
$G\ e\ w \rightarrow \boxed{G(U, V)} \rightarrow \boxed{G(1, H)} \rightarrow G\ e\ A \rightarrow \boxed{h} \rightarrow A$	
$= G\ e\ w \rightarrow \boxed{G(U, VH)} \rightarrow G\ e\ A \rightarrow \boxed{h} \rightarrow A$ taking $Q \triangleq G(U, V)$; bifunctors: $G(U, V)G(1, H) = G(U, VH)$ — relator-laws	
$\sqsubseteq G\ e\ w \rightarrow \boxed{G(U, HR)} \rightarrow G\ e\ A \rightarrow \boxed{h} \rightarrow A$ assumption on V : $Vh \sqsubseteq HR$	
$= G\ e\ w \rightarrow \boxed{G(1, H)} \rightarrow \boxed{G(U, R)} \rightarrow G\ e\ A \rightarrow \boxed{h} \rightarrow A$ bifunctors: $G(U, HR) = G(1, H)G(U, R)$ — relator-laws	
$\sqsubseteq G\ e\ w \rightarrow \boxed{G(1, H)} \rightarrow G\ e\ A \rightarrow \boxed{h} \rightarrow \boxed{R} \rightarrow A$ assumption on h : $G(U, R)h \sqsubseteq hR$	

§15.3. The string edit problem

$Op ::= \text{cpy Char} \mid \text{del Char} \mid \text{ins Char}, F(A, B) = 1 + (A \times B), \alpha \triangleq [\text{nil}, \text{cons}]$.

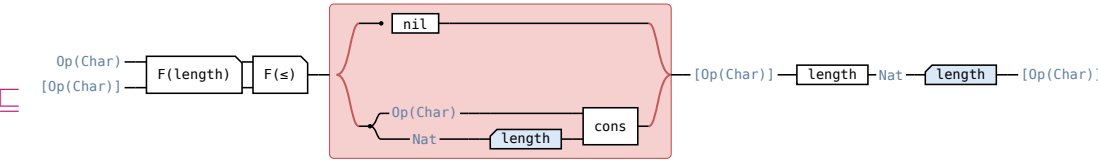
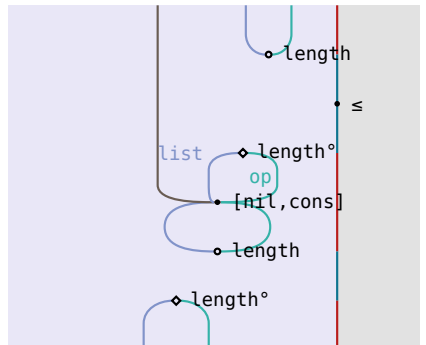
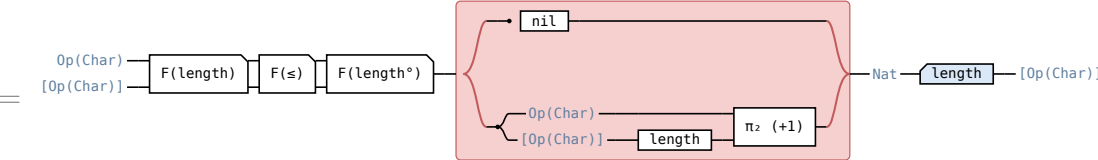
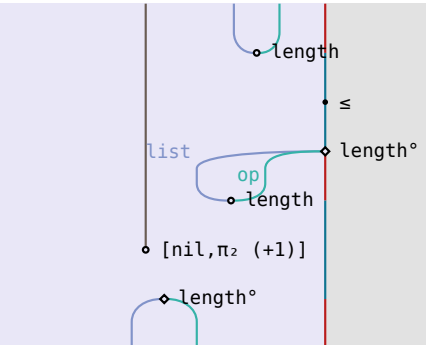
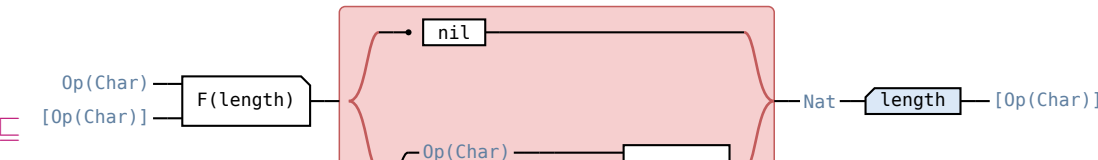
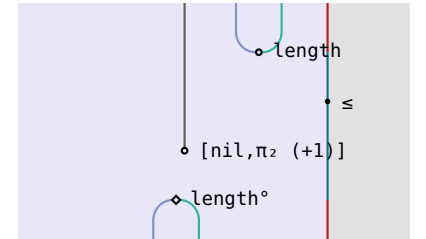
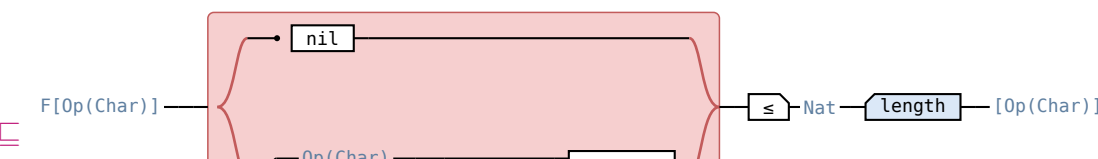
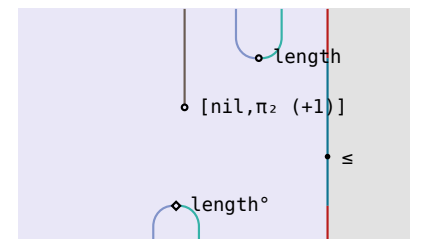
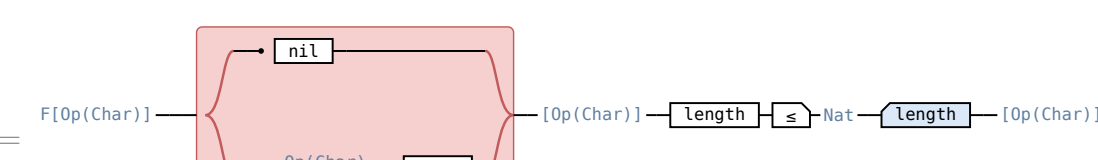
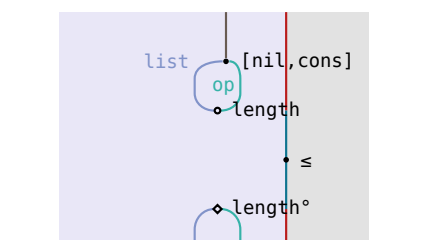
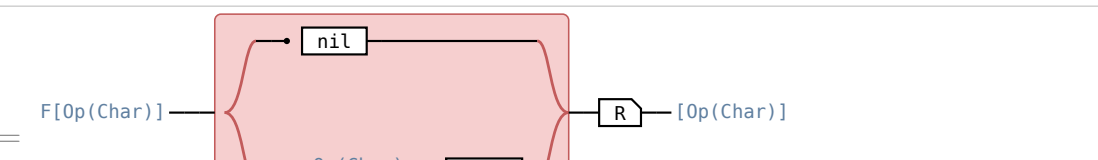
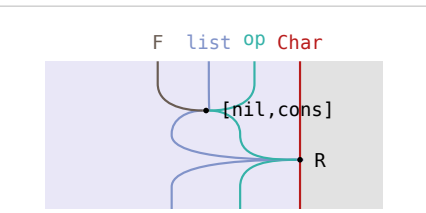
$\text{edit} \triangleq ([\text{base}, \text{step}]) : [Op] \rightarrow [\text{Char}] \times [\text{Char}]$, base returning $([], [])$, step (cpy $a, (xs, ys)$) = $([a] \# xs, [a] \# ys)$, step (del $a, (xs, ys)$) = $([a] \# xs, ys)$, step (ins $a, (xs, ys)$) = $(xs, [a] \# ys)$.

$\text{length} \triangleq ([\text{zero}, \pi_2 \text{ succ}])$, $R \triangleq \text{length} \leq \text{length}^\circ$, $U \triangleq \top$, $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, $Q \triangleq 1 + (U \times V)$, empty the coreflexive on (xs, ys) with both lists empty.

unstep implements $\frac{\text{step}^\circ}{\supset} \text{thin}(U \times V) : \text{unstep } ([a] \# xs, []) = [(\text{del } a, (xs, []))]$, $\text{unstep } ([], [b] \# ys) = [(\text{ins } b, ([], ys))]$, $\text{unstep } ([a] \# xs, [b] \# ys) = (a = b \rightarrow [(\text{cpy } a, (xs, ys))], [(\text{del } a, (xs, [b] \# ys)), (\text{ins } b, ([a] \# xs, ys))])$.

$(\mu X : \frac{\text{editAlg}^\circ}{3} \text{ thin}(Q)P(F(X)[\text{nil}, \text{cons}])\text{est}(R)) \sqsubseteq \frac{\text{edit}^\circ}{3} \text{ est}(R) , \text{ mle} = (\text{empty} \rightarrow \text{nil}, \text{unstep list}((1 \times \text{mle}) \text{cons}) \text{minlist}(R))$ a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed	
circuit	Hinze–Marsden
$\begin{array}{c} [\text{Char}] \\ [\text{Char}] \end{array} \rightarrow \frac{1}{3} \rightarrow E[\text{Char}]^2 \rightarrow E(\text{edit}^\circ) \rightarrow E[Op(\text{Char})] \rightarrow \text{est}(R) \rightarrow [Op(\text{Char})]$ the specification $\triangleq \text{edit}$	
$\begin{array}{c} [\text{Char}] \\ [\text{Char}] \end{array} \rightarrow \frac{1}{3} \rightarrow E[\text{Char}]^2 \rightarrow E(\text{editAlg}^\circ) \rightarrow \text{thin}(Q) \rightarrow E[\text{Char}]^2 \rightarrow P(F(X)[\text{nil}, \text{cons}]) \rightarrow E[Op(\text{Char})] \rightarrow \text{est}(R) \rightarrow [Op(\text{Char})]$ at $Q \triangleq 1 + (U \times V)$. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is Proposition 9.2 at $\text{length} \triangleq ([\text{zero}, \pi_2 \text{succ}])$ with succ monotonic on $\leq$, so cons is monotonic on R. The thinning condition $QF(1, \text{edit}^\circ) \alpha \sqsubseteq F(1, \text{edit}^\circ) \alpha R$ over $F(Op, [\text{Char}] \times [\text{Char}])$ is Proposition 9.4 at $U \triangleq T - F(T, R) \alpha \sqsubseteq \alpha R$, left as an exercise, so any two operations may be compared — and $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, — $\text{edit} (1 \times \text{suffix}) \sqsubseteq R^\circ \text{edit}$, $\text{edit} (\text{suffix} \times 1) \sqsubseteq R^\circ \text{edit}$. $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \Rightarrow BA^* \sqsubseteq C^*B$ cut those to one step each: drop the operation that produced the head, or weaken its cpy to a del, never lengthening the sequence	
$\begin{array}{c} [\text{Char}] \\ [\text{Char}] \end{array} \rightarrow \frac{1}{3} \rightarrow E[\text{Char}]^2 \rightarrow E(\text{step}^\circ) \rightarrow \text{thin}(T \times V) \rightarrow E(Op(\text{Char}) \times [\text{Char}] \times [\text{Char}]) \rightarrow P((1 \times X) \text{cons}) \rightarrow E[Op(\text{Char})] \rightarrow \text{est}(R) \rightarrow [Op(\text{Char})]$ $\equiv$ Proposition 9.1: base and step have disjoint ranges, so the fixed point splits into one branch per summand — empty, the coreflexive on (xs, ys) with both lists empty, is where base returns	
$\begin{array}{c} [\text{Char}] \\ [\text{Char}] \end{array} \rightarrow \text{unstep} \rightarrow [Op(\text{Char}) \times [\text{Char}] \times [\text{Char}]] \rightarrow \text{list}((1 \times \text{mle}) \text{cons}) \rightarrow [Op(\text{Char})] \rightarrow \text{minlist}(R) \rightarrow [Op(\text{Char})]$ $\equiv$ unstep implements $\frac{\text{step}^\circ}{3} \text{ thin}(U \times V)$ — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and minlist(R) implements est(R). The same subproblem is solved many times over, so the running time is exponential in the two lengths	

$F(R)[\text{nil}, \text{cons}] \sqsubseteq [\text{nil}, \text{cons}]R$ comparing the tails by R and then putting an operation in front ($F(R) \alpha$) relates only sequences that R also relates once the operation is in front (αR)	
circuit	Hinze–Marsden
$\begin{array}{c} F[Op(\text{Char})] \\ [Op(\text{Char})] \end{array} \rightarrow \left\{ \begin{array}{l} \text{nil} \\ Op(\text{Char}) \rightarrow [Op(\text{Char})] \rightarrow R \rightarrow \text{cons} \end{array} \right\} \rightarrow [Op(\text{Char})]$	
$\begin{array}{c} Op(\text{Char}) \\ [Op(\text{Char})] \end{array} \rightarrow F(\text{length}) \rightarrow F(\leq) \rightarrow \left\{ \begin{array}{l} \text{nil} \\ Op(\text{Char}) \rightarrow \text{Nat} \rightarrow \text{length} \rightarrow \text{cons} \end{array} \right\} \rightarrow [Op(\text{Char})]$ $R \triangleq \text{length} \leq \text{length}^\circ \triangleq \text{edit}$; F preserves composition	

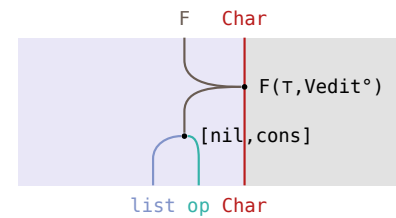
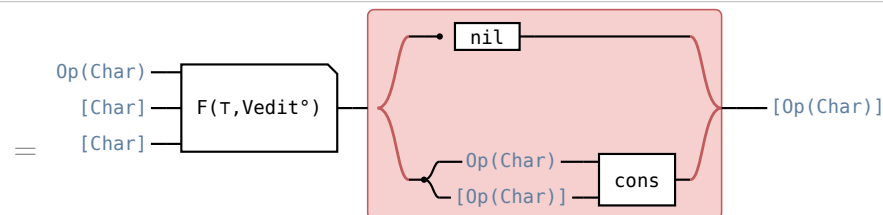
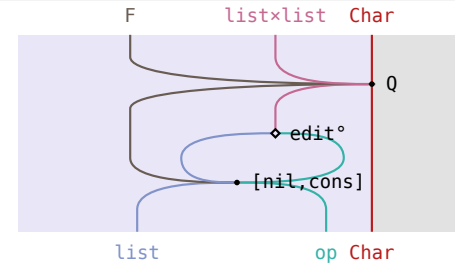
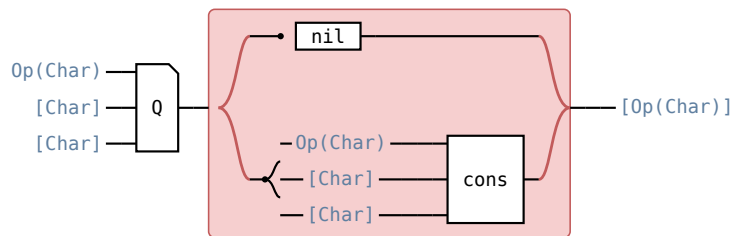
circuit	Hinze–Marsden
 length is a map, so entire: $1 \sqsubseteq \text{length} \text{ length}^\circ$	
 $\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}]: \text{length} \triangleq ([\text{zero}, \pi_2 \text{ succ}]) \text{ — edit}$	
 length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1$	
 succ is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq$	
 $F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}$ again	
 $R \triangleq \text{length} \leq \text{length}^\circ \text{ — edit}$	

$$QF(\text{edit}^\circ)[\text{nil}, \text{cons}] \sqsubseteq F(\text{edit}^\circ)[\text{nil}, \text{cons}]R$$

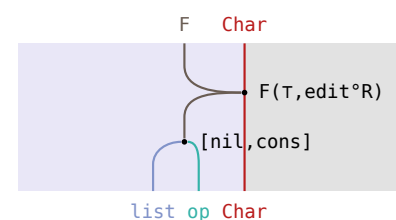
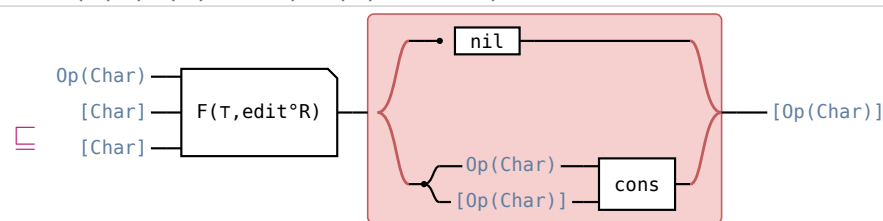
every edit sequence built from a decomposition Q puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one

circuit

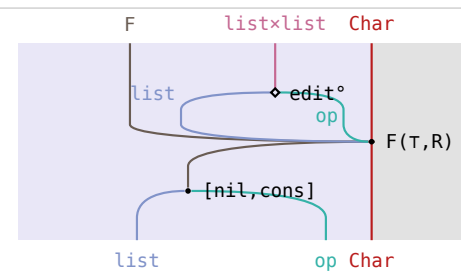
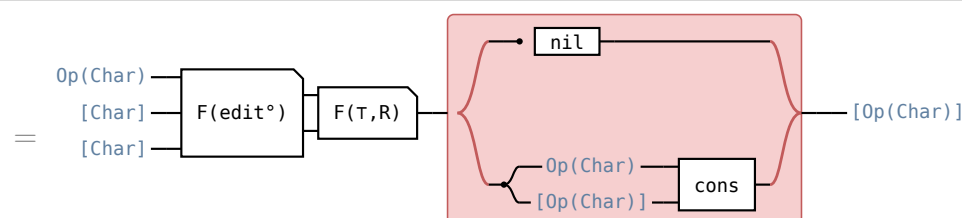
Hinze–Marsden



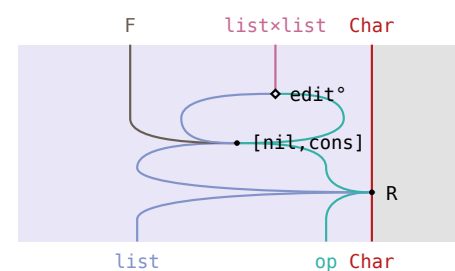
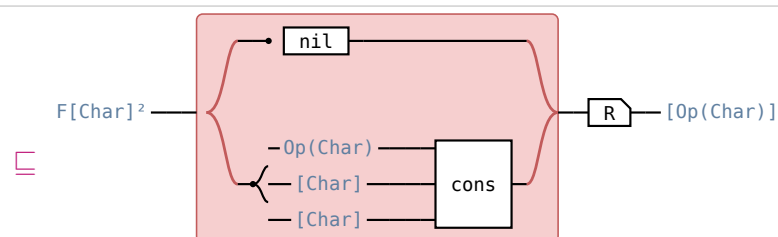
$Q \triangleq 1 + (U \times V)$ is $F(U, V)$ at $U \triangleq \tau$ — $\triangleq \text{edit}$; and the bifunctor F preserves composition:
 $F(U, V)F(1, \text{edit}^\circ) = F(U, V \text{ edit}^\circ)$



Proposition 9.4's second condition $V \text{ edit}^\circ \sqsubseteq \text{edit}^\circ R$ — (15.3e)



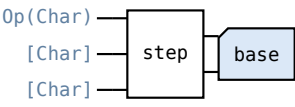
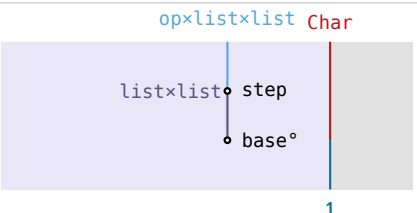
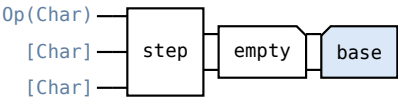
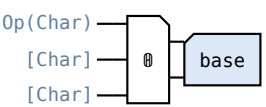
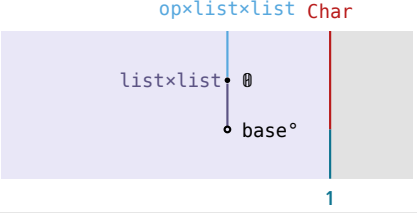
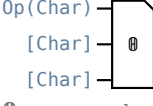
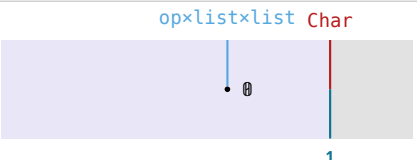
F preserves composition: $F(U, \text{edit}^\circ R) = F(1, \text{edit}^\circ)F(U, R)$



Proposition 9.4's first condition $F(\tau, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: **CONS** adds one to both lengths whatever the two operations are

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
$=$ $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	
$\sqsubseteq$ $\text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins. Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	
$\sqsubseteq$ $\text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	
$\sqsubseteq$ R° is transitive, $\leq$ being so	

(15.3f)

$\text{step } \text{base}^\circ = \emptyset$ no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

(15.3g)

$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)] \mid \text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left	
$\text{column}(xs, \text{cons}(b, ys)) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)] \mid xus), xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right	
$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $= \text{step}(b)((a, (u, v)), ws) = (a = b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths	

§15.4. Optimal bracketing

(15.4a)

tree $A ::= \text{tip } A \mid \text{bin } (\text{tree } A, \text{tree } A)$, $FX = A + X^2$, so $F(R) = 1 + R^2$; $h \triangleq [\text{tip}, \text{bin}]$, $\text{flatten} \triangleq ([\text{wrap}, \text{cat}] \mid) : \text{tree } A \rightarrow \text{list}^+ A$, $H = \text{flatten}^\circ$.

$\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}] \mid)$, $\text{opt} \triangleq (\text{zero}, \text{st})$, $\text{opb } ((cx, sx), (cy, sy)) = (\text{cb } (sx, sy) + cx + cy, \text{sb } (sx, sy))$.

sb associative, so $\text{size} = \text{flatten } \text{sz}$ for a map $\text{sz} : R \triangleq \text{cost} \leq \text{cost}^\circ$, $g \triangleq [\text{zero}, (1 \times \text{sz})^2 \text{ opb } \pi_1]$, single the coreflexive on singleton lists.

$\text{splits} \triangleq (\text{inits}^+, \text{tails}^+) \text{ zip}$, an implementation of $\frac{\text{cat}^\circ}{\exists}$; $\text{array} \triangleq \text{inits } \text{list}(\text{row})$, $\text{row} \triangleq \text{tails } \text{list}(\text{mct})$, $\text{col} \triangleq \text{inits } \text{list}(\text{mct})$.

$\text{mix} \triangleq \text{zip } \text{list}(\text{bin}) \text{ minlist}(R)$, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}$, $\text{process} \triangleq ((\text{tip } \text{wrap}) \times 1) \text{ loop}(\text{next})$.

$\text{Assoc}(\text{sb}) \Rightarrow F(\langle \text{cost}, \text{flatten} \rangle)g = [\text{tip}, \text{bin}] \text{cost}$ pairing each subtree with its cost and its flattening and then applying g gives what building the tree by $[\text{tip}, \text{bin}]$ and taking its cost gives	
circuit	Hinze–Marsden
$F(\text{tree}(A)) \rightarrow F(\langle \text{cost}, \text{flatten} \rangle) \rightarrow F(\text{Int} \times \text{list}^+(A)) \rightarrow g \rightarrow \text{Int}$	
$A + \text{tree}(A)^2 \rightarrow \text{zero} \rightarrow \text{cost} \rightarrow \text{flatten} \rightarrow \text{list}^+(A) \rightarrow \text{sz} \rightarrow \text{opb} \rightarrow \text{Int}$ definition of g; coproducts and products — $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \text{zero} \rightarrow \text{cost} \rightarrow \text{size} \rightarrow \text{opb} \rightarrow \text{Int}$ $\text{flatten sz} = \text{size}$, since sb is associative — $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \text{zero} \rightarrow \text{bin} \rightarrow \text{cost} \rightarrow \text{size} \rightarrow \text{opb} \rightarrow \text{Int}$ $\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, at a node $\text{bin}(\text{cost}, \text{size}) = (\text{cost}, \text{size})^2 \text{ opb}$	
$A + \text{tree}(A)^2 \rightarrow \text{tip} \rightarrow \text{tree}(A) \rightarrow \text{cost} \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \text{cost} \rightarrow \text{Int}$ $\text{tip cost} = \text{zero}$; products, $\langle \text{cost}, \text{size} \rangle \pi_1 = \text{cost}$	
$A + \text{tree}(A)^2 \rightarrow \text{tip} \rightarrow \text{tree}(A) \rightarrow \text{cost} \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \text{cost} \rightarrow \text{Int}$ coproducts, $[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{cost}$	

$F(\leq \times 1)g \leq g \leq$ raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $=$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$	
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	

Assoc(sb) $\Rightarrow$ ($\mu X : \frac{[\text{wrap}, \text{cat}]^\circ}{3} P([\text{tip}, (X \times X) \text{bin}]) \text{est}(R) \sqsubseteq \frac{\text{flatten}^\circ}{3} \text{est}(R)$, $\text{mct} = (\text{single-head tip}, (\text{init col}, \text{tail row}) \text{mix})$ a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{\frac{1}{3}} E(\text{list}^+(A)) \xrightarrow{E(\text{flatten}^\circ)} E(\text{tree}(A)) \xrightarrow{\text{est}(R)} \text{tree}(A)$ the specification $\text{---} \triangleq \text{flatten}$	
$\text{list}^+(A) \xrightarrow{\frac{1}{3}} E(\text{list}^+(A)) \xrightarrow{E([\text{wrap}, \text{cat}]^\circ)} E F(\text{list}^+(A)) \xrightarrow{P([\text{tip}, (X \times X) \text{bin}])} E(\text{tree}(A)) \xrightarrow{\text{est}(R)} \text{tree}(A)$ split the list in every way, bracket both halves, join. The condition is monotonicity in context, $F(Rn(\text{flatten} \text{ flatten}^\circ)) h \sqsubseteq h R$ — only trees with the same flattening are compared — which is Proposition 9.3 at $H^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip}, \text{bin}]$ $\text{cost} = (1 + (\text{cost}, \text{flatten})^2) g$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq 1)^2) g \sqsubseteq g$ (g monotonic on $\leq$ in its two cost arguments)	
$\text{list}^+(A) \xrightarrow{\frac{1}{3}} E(\text{list}^+(A)) \xrightarrow{E(\text{cat}^\circ)} E(\text{list}^+(A)^2) \xrightarrow{P((X \times X) \text{bin})} E(\text{tree}(A)) \xrightarrow{\text{est}(R)} \text{tree}(A)$ $\equiv$ Proposition 9.1: wrap and cat have disjoint ranges, and single is the coreflexive on singleton lists, where wrap returns	
$\text{list}^+(A) \xrightarrow{\text{splits}} [\text{list}^+(A)^2] \xrightarrow{\text{list}((\text{mct} \times \text{mct}) \text{bin})} [\text{tree}(A)] \xrightarrow{\text{minlist}(R)} \text{tree}(A)$ $\sqsubseteq$ $\text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$ zip implements $\frac{\text{cat}^\circ}{3}$ and $\text{minlist}(R)$ implements $\text{est}(R)$. Exponential, since the segments of one list overlap	
$\text{mct} = (\text{single-head tip}, (\text{init col}, \text{tail row}) \text{mix})$ (9.7) $\equiv$ the tabulation: $\text{mct } xS$ is needed for every non-empty segment xS, so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{minlist}(R)$	
$\text{col} = (\text{single-head tip wrap}, (\text{init col}, \text{tail row}) \text{next})$ (9.8) $\text{cons col} = (1 \times \text{array}) \text{process}$ (9.9) $\equiv$ $\text{row} = (\text{single-head tip wrap}, (\text{mct}, \text{tail row}) \text{cons})$ (9.10) a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq (\pi_1, \text{mix}) \text{snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{loop}(\text{next})$	
$\text{array} = ([\text{fstcol}, \text{addcol}]), \text{fstcol} \triangleq \text{tip wrap wrap}$ $\equiv$ $\text{addcol} \triangleq (\pi_1 \text{tip wrap}, \text{step}) \text{cons}, \text{step} \triangleq (\text{process tail}, \pi_2) \text{zip list}(\text{cons})$ the program: one fold building the array column by column, cubic in the length of the input	

(15.4e)

nonsingle mct=nonsingle(init col,tail row)mix

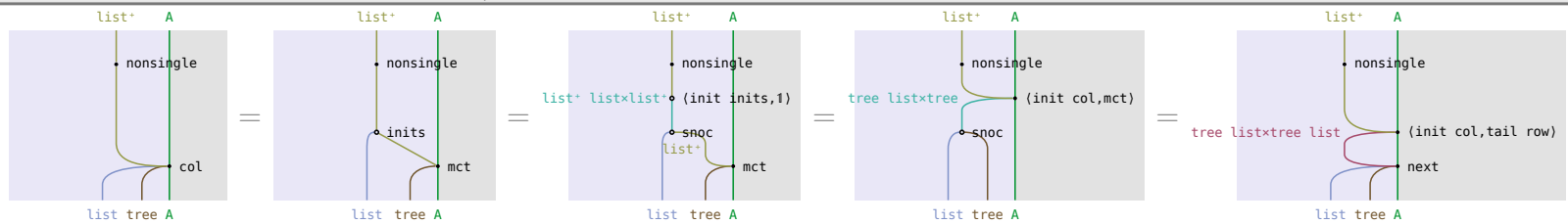
on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



$$\begin{aligned}
 & \text{list}^+(A) \text{--- nonsingle --- mct --- tree}(A) \\
 = & \text{list}^+(A) \text{--- nonsingle --- splits --- [list}^+(A)^2 \text{] --- list(graft) --- [tree}(A) \text{] --- minlist}(R) \text{--- tree}(A) \quad \text{recursive case of mct} \triangleq \text{flatten} \\
 = & \text{list}^+(A) \text{--- nonsingle --- } \begin{cases} \text{inits}^+ \\ \text{tails}^+ \end{cases} \text{--- [list}^+(A) \text{] --- zip --- [list}^+(A)^2 \text{] --- list(graft) --- [tree}(A) \text{] --- minlist}(R) \text{--- tree}(A) \\
 & \text{definition of splits} \triangleq \text{flatten} \\
 = & \text{list}^+(A) \text{--- nonsingle --- trees --- zip --- [tree}(A)^2 \text{] --- list(bin) --- [tree}(A) \text{] --- minlist}(R) \text{--- tree}(A) \\
 & \text{definitions of graft} \triangleq (\text{mct} \times \text{mct}) \text{ bin and trees} \triangleq (\text{inits}^+ \text{ list}(\text{mct}), \text{tails}^+ \text{ list}(\text{mct})), \text{our names; zip} \\
 & \text{list}(f \times g) = (\text{list}(f) \times \text{list}(g)) \text{ zip} \\
 = & \text{list}^+(A) \text{--- nonsingle --- trees --- mix --- tree}(A) \quad \text{introducing mix} \triangleq \text{zip list(bin) minlist}(R) \triangleq \text{flatten} \\
 = & \text{list}^+(A) \text{--- nonsingle --- } \begin{cases} \text{init} & \text{col} \\ \text{tail} & \text{row} \end{cases} \text{--- [tree}(A) \text{] --- mix --- tree}(A) \\
 & \text{definition of trees; inits}^+ = \text{init inits and tails}^+ = \text{tail tails on non-singletons; definition of row and col} \triangleq \text{flatten}
 \end{aligned}$$

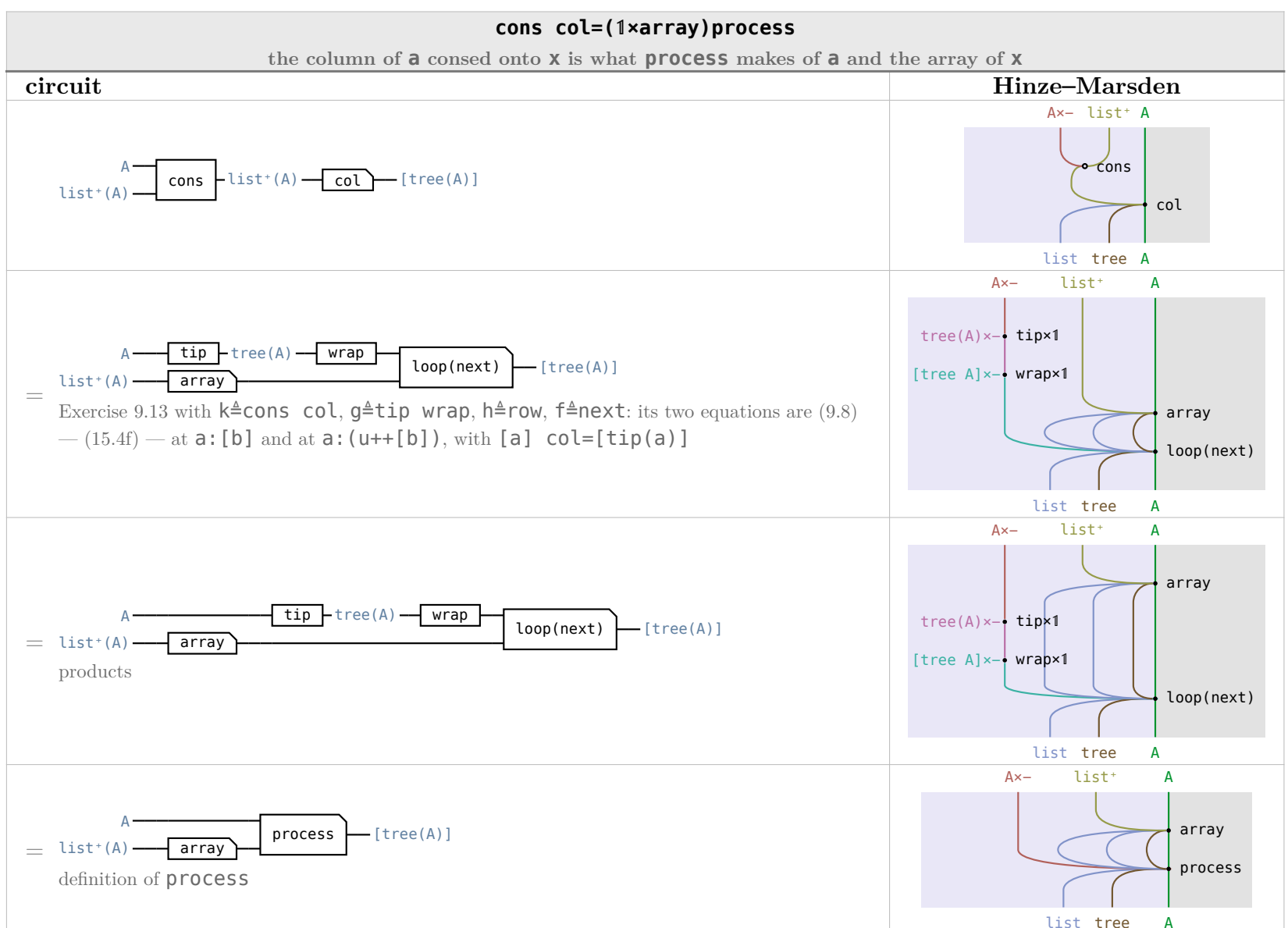
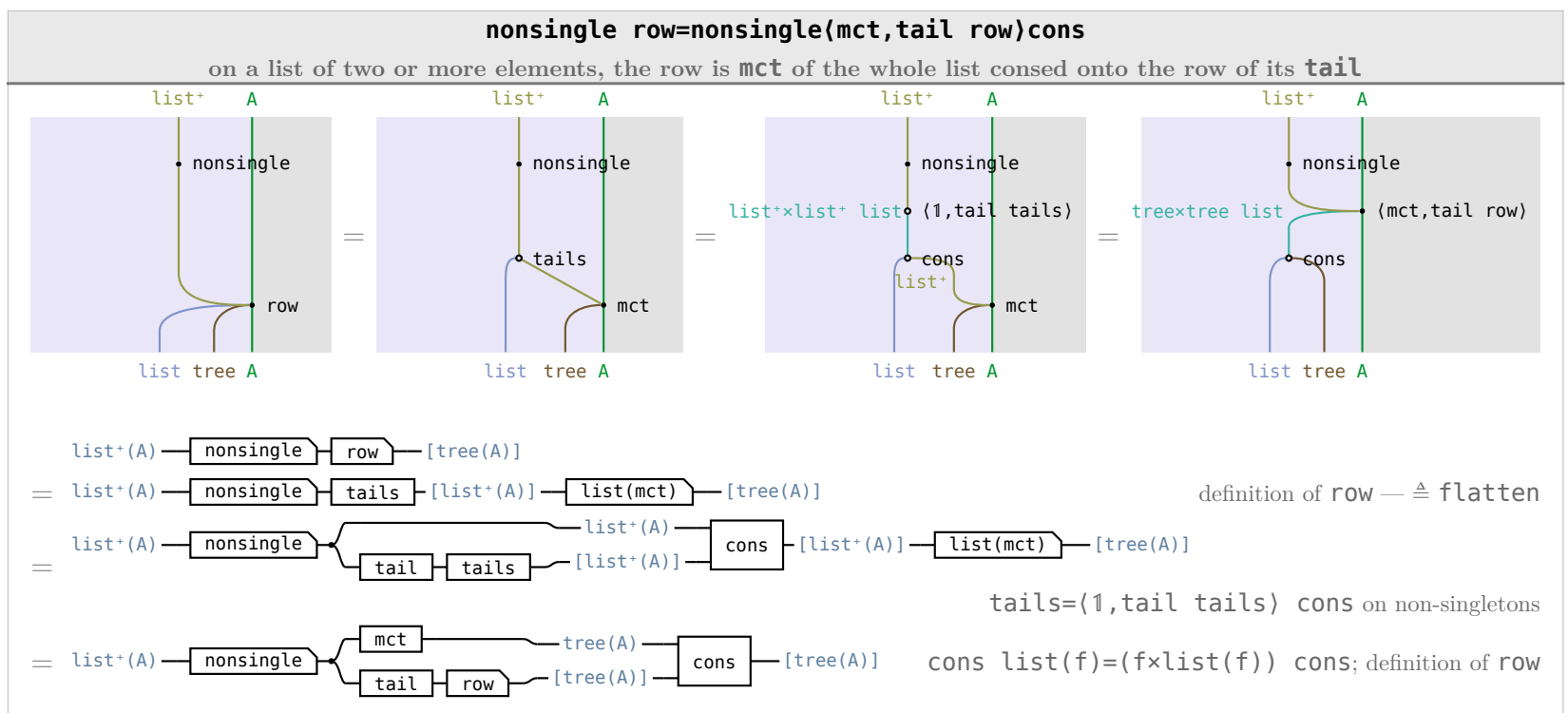
nonsingle col=nonsingle(init col,tail row)next

on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**

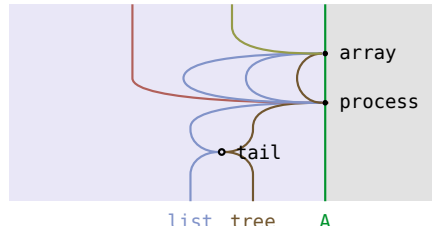


$$\begin{aligned}
 & \text{list}^+(A) \text{--- nonsingle --- col --- [tree}(A) \text{]} \\
 = & \text{list}^+(A) \text{--- nonsingle --- inits --- [list}^+(A) \text{] --- list(mct) --- [tree}(A) \text{]} \quad \text{definition of col} \triangleq \text{flatten} \\
 = & \text{list}^+(A) \text{--- nonsingle --- } \begin{cases} \text{init} & \text{inits} \\ & \text{list}^+(A) \end{cases} \text{--- snoc --- [list}^+(A) \text{] --- list(mct) --- [tree}(A) \text{]} \\
 & \text{inits} = (\text{init inits}, 1) \text{ snoc on non-singletons} \\
 = & \text{list}^+(A) \text{--- nonsingle --- } \begin{cases} \text{init} & \text{col} \\ \text{mct} & \text{tree}(A) \end{cases} \text{--- snoc --- [tree}(A) \text{]} \quad \text{snoc list}(f) = (\text{list}(f) \times f) \text{ snoc; definition of col} \\
 = & \text{list}^+(A) \text{--- nonsingle --- } \begin{cases} \text{init} & \text{col} \\ \text{tail} & \text{row} \end{cases} \text{--- [tree}(A) \text{] --- next --- [tree}(A) \text{]} \\
 & (9.7) \text{ on non-singletons} \text{--- (15.4e) --- and introducing next} \triangleq (\pi_1, \text{mix}) \text{ snoc} \triangleq \text{flatten}
 \end{aligned}$$

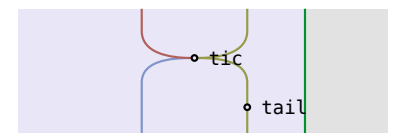
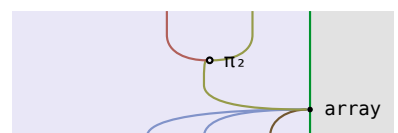
(15.4f)



(15.4i)

tops=(1×array)process tail the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
	
$= \text{list}^+(A) \text{---} \boxed{\text{cons}} \text{---} \text{list}^+(A) \text{---} \boxed{\text{inits}} \text{---} \boxed{\text{tail}} \text{---} [\text{list}^+(A)] \text{---} \boxed{\text{list(mct)}} \text{---} [\text{tree}(A)]$ definition of tops$\triangleq$tic list(mct), our name	
$= \text{list}^+(A) \text{---} \boxed{\text{cons}} \text{---} \text{list}^+(A) \text{---} \boxed{\text{col}} \text{---} \boxed{\text{tail}} \text{---} [\text{tree}(A)]$ tail list(f)=list(f) tail; definition of col	
$= \text{list}^+(A) \text{---} \boxed{\text{array}} \text{---} \boxed{\text{process}} \text{---} \boxed{\text{tail}} \text{---} [\text{tree}(A)]$ (9.9) — (15.4h)	

(15.4j)

rests=π_2 array the rest of each new row is the array of x	
circuit	Hinze–Marsden
	
$= \text{list}^+(A) \text{---} \boxed{\text{cons}} \text{---} \text{list}^+(A) \text{---} \boxed{\text{inits}} \text{---} \boxed{\text{tail}} \text{---} [\text{list}^+(A)] \text{---} \boxed{\text{list(tail row)}} \text{---} [[\text{tree}(A)]]$ definition of rests$\triangleq$tic list(tail row), our name	
$= \text{list}^+(A) \text{---} \boxed{\text{array}} \text{---} [[\text{tree}(A)]]$ tic list(tail)=π_2 inits; definition of array	

$\text{newrows} = (1 \times \text{array}) \text{step}$ the rows of the array of $\mathbf{a}:\mathbf{x}$ below its first are what step makes of $\mathbf{a}$ and the array of $\mathbf{x}$	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{newrows}} \rightarrow [[\text{tree}(A)]]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{init}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list}(\text{row})} [[\text{tree}(A)]]$ definition of $\text{newrows} \triangleq \text{tic list}(\text{row})$, our name	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{init}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list}(\text{mct}, \text{tail row}) \text{cons}} [[\text{tree}(A)]]$ (9.10) — (15.4g) — on the non-singleton tic lists	
$= \text{list}^+(A) \xrightarrow{A} \begin{cases} \text{tops} \\ \text{rests} \end{cases} \rightarrow \begin{cases} [\text{tree}(A)] \\ [[\text{tree}(A)]] \end{cases} \xrightarrow{\text{zip}} [\text{tree}(A) \times [\text{tree}(A)]] \xrightarrow{\text{list}(\text{cons})} [[\text{tree}(A)]]$ $\text{list}(f, g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{array}} \xrightarrow{\text{step}} [[\text{tree}(A)]]$ (15.4i) and (15.4j); products; definition of step	

cons array=(1×array)addcol the array of a consed onto x is what addcol makes of a and the array of x	
circuit	Hinze–Marsden
$= \text{list}^+(A) \xrightarrow{\text{cons}} \text{list}^+(A) \xrightarrow{\text{inits}} [\text{list}^+(A)] \xrightarrow{\text{list}(\text{row})} [[\text{tree}(A)]]$ definition of array	
$= \text{cons } \text{inits} = (\pi_1 \text{ wrap, tic}) \text{ cons, abbreviating cons inits tail by tic}$	
$= \text{list}^+(A) \xrightarrow{\text{newrows}} \text{list}^+(A) \xrightarrow{\text{wrap}} \text{list}^+(A) \xrightarrow{\text{row}} [\text{tree}(A)] \xrightarrow{\text{cons}} [[\text{tree}(A)]]$ cons list(f)=(f×list(f)) cons; definition of newrows	
$= \text{list}^+(A) \xrightarrow{\text{newrows}} \text{list}^+(A) \xrightarrow{\text{tip}} \text{tree}(A) \xrightarrow{\text{wrap}} [\text{tree}(A)] \xrightarrow{\text{cons}} [[\text{tree}(A)]]$ wrap row=tip wrap	
$= \text{list}^+(A) \xrightarrow{\text{array}} \text{array} \xrightarrow{\text{addcol}} [[\text{tree}(A)]]$ (15.4k); products; definition of addcol	

§15.5. Data compression

$[A] ::= \text{nil} \mid \text{snoc } ([A], A), \text{ list}^+ A ::= \text{wrap } A \mid \text{snoc } (\text{list}^+ A, A), \text{ String} = [\text{Char}], \text{ Code} ::= \text{sym } \text{Char} \mid \text{ptr } (\text{String}, \text{String}^+), \text{ F}(\text{Code}, \text{String}) = 1 + (\text{String} \times \text{Code}), \alpha \triangleq [\text{nil}, \text{snoc}].$

$\text{decode} \triangleq ([\text{nil}, \text{extend}]) : [\text{Code}] \rightarrow \text{String}, \text{ extend } (xs, \text{sym } a) = xs \# [a], \text{ extend } (xs, \text{ptr } (ys, zs)) = xs \# zs \text{ when } ys \# zs \text{ is a proper prefix of } xs \# zs; \text{ H} = \text{decode}^\circ.$

$\text{size} \triangleq ([\text{zero}, \text{distr } [1 \times c, 1 \times p] \text{ plus}])$ with c, p the constant costs of a symbol and a pointer; $R \triangleq \text{size} \leq \text{size}^\circ$, $Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers.

$\text{lrt } ws = \text{est}(\text{prefix}^\circ \times (\tau + \tau)) \{ (xs, (ys, zs)) \mid xs \# zs = ws, ys \# zs \text{ a proper prefix of } ws \}$, the longest repeated tail; $\text{reduce } (ws \# [a]) = (zs \neq [] \rightarrow [(ws, \text{sym } a), (xs, \text{ptr } (ys, zs))], [(ws, \text{sym } a)])$ with $(xs, (ys, zs)) = \text{lrt } (ws \# [a])$.

$$0 \leq c \wedge 0 \leq p \Rightarrow (\mu X : \frac{[\text{nil}, \text{extend}]^\circ}{3} \text{thin}(Q) P([\text{nil}, (X \times 1) \text{snoc}]) \text{est}(R)) \sqsubseteq \frac{\text{decode}^\circ}{3} \text{est}(R), \text{ encode} = (\text{null} \rightarrow \text{nil}, \text{reduce} \text{list}((\text{encode} \times 1) \text{snoc}) \text{minlist}(R))$$

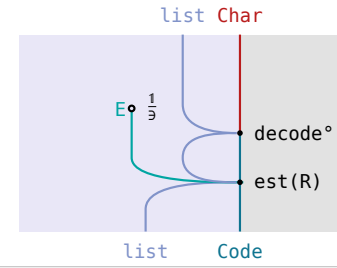
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit

$$[\text{Char}] \xrightarrow{\frac{1}{3}} E[\text{Char}] \xrightarrow{E(\text{decode}^\circ)} E[\text{Code}] \xrightarrow{\text{est}(R)} [\text{Code}]$$

the specification $\triangleq \text{decode}$

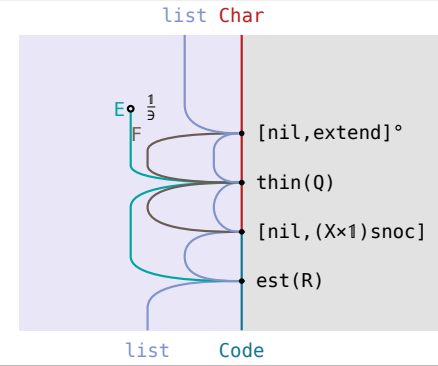
Hinze–Marsden



$$[\text{Char}] \xrightarrow{\frac{1}{3}} E[\text{Char}] \xrightarrow{E([\text{nil}, \text{extend}]^\circ)} \text{thin}(Q) \xrightarrow{E[\text{Char}]} P([\text{nil}, (X \times 1) \text{snoc}]) \xrightarrow{E[\text{Code}]} \text{est}(R) \xrightarrow{[\text{Code}]}$$

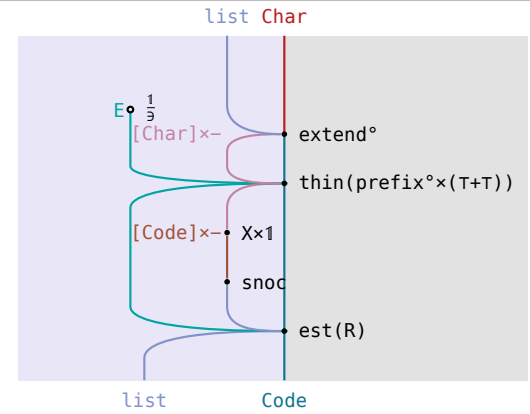
at $Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(\tau + \tau)[c, p] = [c, p]$.

Proposition 9.4 splits the thinning condition in two — $F(\tau + \tau, R) \alpha \sqsubseteq \alpha R$, and $\text{decode prefix} \sqsubseteq R^\circ \text{decode}$, for which $\text{decode init} \sqsubseteq R^\circ \text{decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



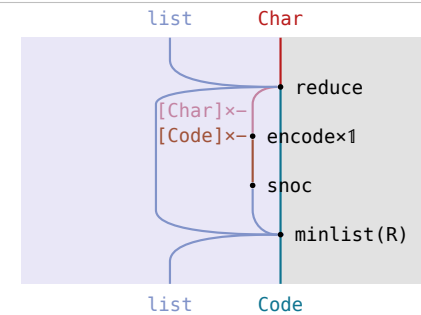
$$[\text{Char}] \xrightarrow{\frac{1}{3}} E[\text{Char}] \xrightarrow{E(\text{extend}^\circ)} \text{thin}(\text{prefix}^\circ \times (\tau + \tau)) \xrightarrow{E([\text{Char}] \times \text{Code})} P((X \times 1) \text{snoc}) \xrightarrow{E[\text{Code}]} \text{est}(R) \xrightarrow{[\text{Code}]}$$

Proposition 9.1: nil and extend have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^\circ}{3} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer



$$[\text{Char}] \xrightarrow{\text{reduce}} [[\text{Char}] \times \text{Code}] \xrightarrow{\text{list}((\text{encode} \times 1) \text{snoc})} [[\text{Code}]] \xrightarrow{\text{minlist}(R)} [\text{Code}]$$

reduce implements $\frac{\text{extend}^\circ}{3} \text{thin}(\text{prefix}^\circ \times (\tau + \tau))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt . Again exponential; the book gives no tabulation for it

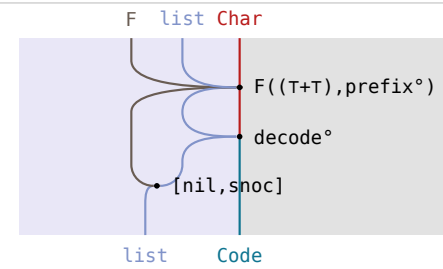
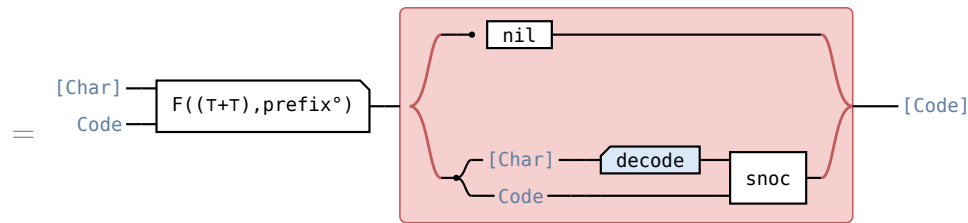
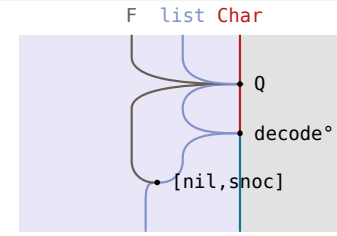
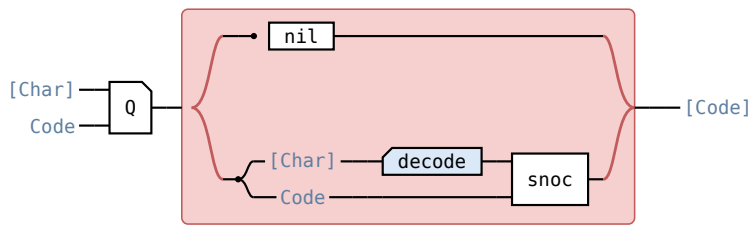


$$0 \leq c \wedge 0 \leq p \Rightarrow QF(\text{decode}^\circ)[\text{nil}, \text{snoc}] \sqsubseteq F(\text{decode}^\circ)[\text{nil}, \text{snoc}]R$$

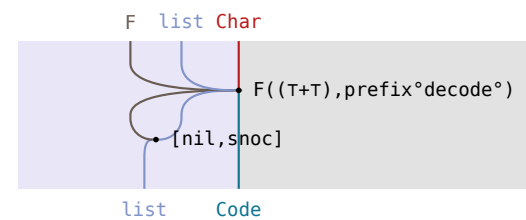
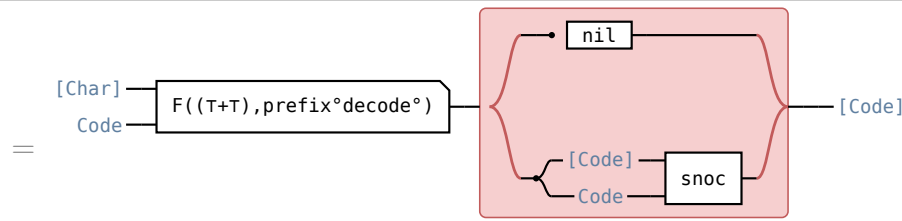
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one

circuit

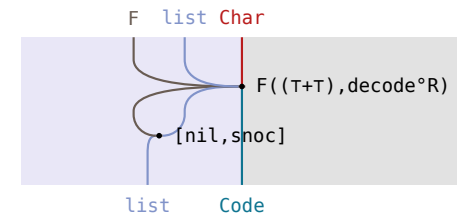
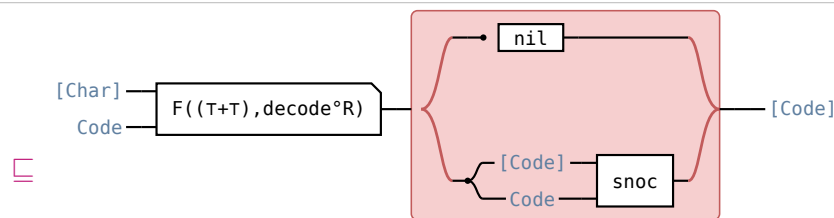
Hinze–Marsden



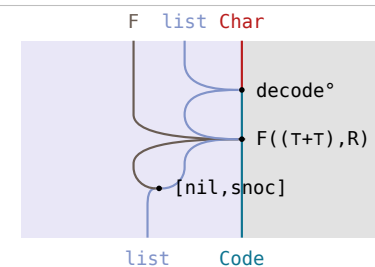
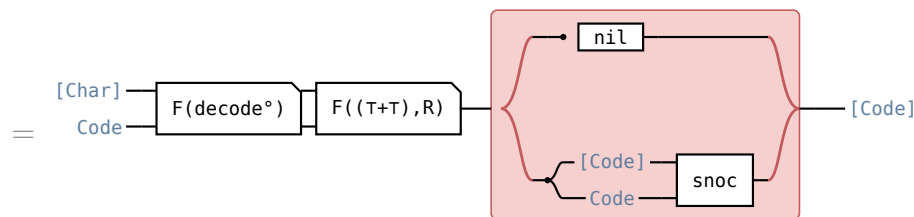
$Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) \triangleq \text{decode}$



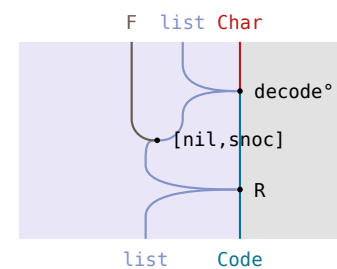
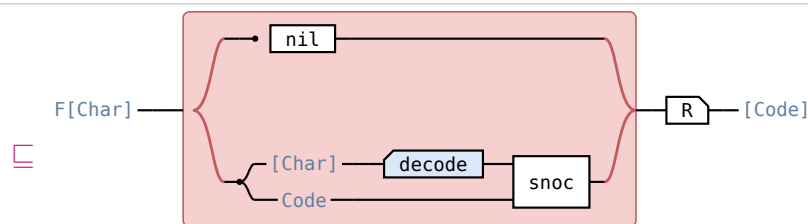
the bifunctor F preserves composition: $F(U, \text{prefix}^\circ)F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)$



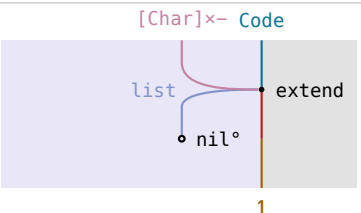
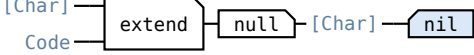
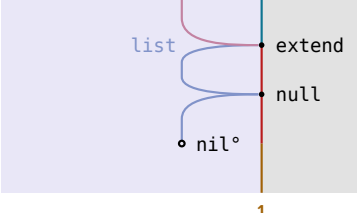
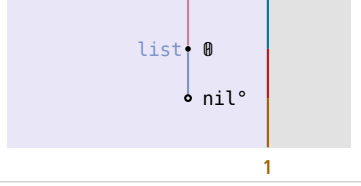
Proposition 9.4's second condition $\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last character drops the last **sym**, or shortens or drops the last pointer. Proved at **prefix** directly by induction, not through **init**



F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ)F(U, R)$



Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: **snoc** adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$

extend $\text{nil}^\circ = \emptyset$ no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
	
 nil returns only [], so $\text{nil}^\circ = \text{null } \text{nil}^\circ$	
 extend null = ∅: a symbol ends the string with a character, a pointer with its non-empty ZS	
 $\emptyset$ composed with anything is $\emptyset$	